# Homework 5 — Rental Housing Listings

Repository: github.com/ishuapurva1996/data260-6102

**Pragya Apurva | SID4 6102 | October 5, 2026**

The application manages rental listings and property managers. Read-only MCP tools expose public data. A shared executor validates calls, retries temporary failures and blocks private-contact requests. A local model uses these tools through a bounded, logged agent loop.

### Part 0 — Personal configuration

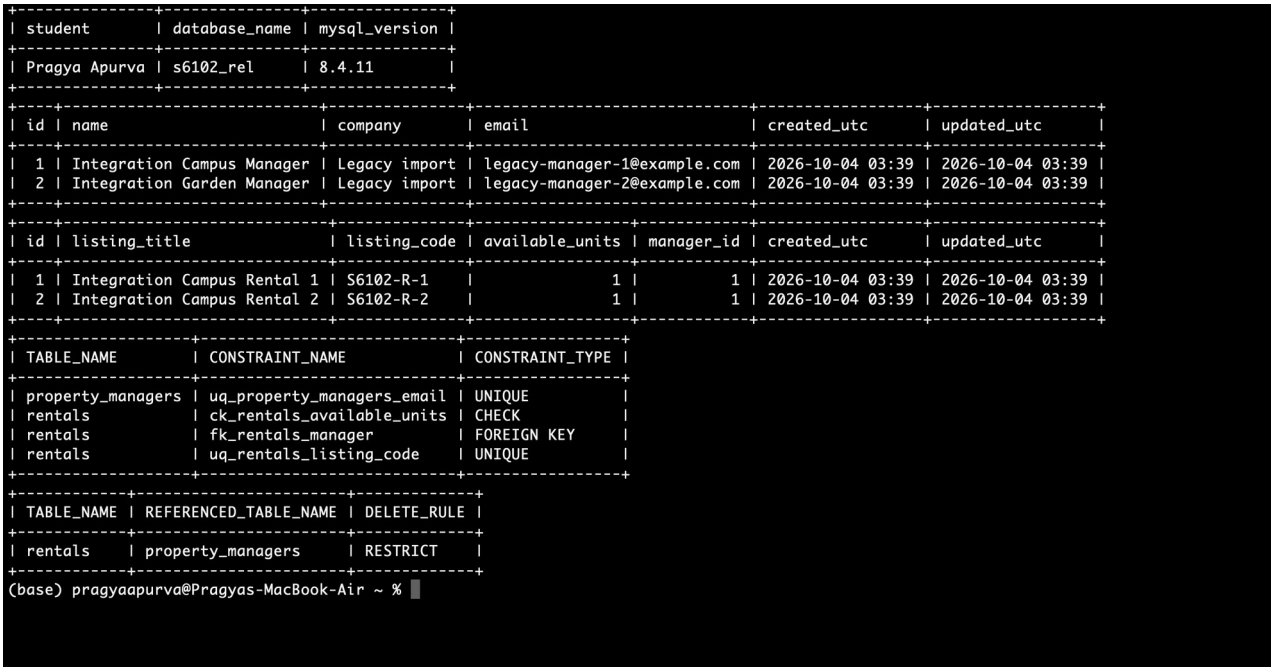
| Setting | Value |
|---|---|
| SID4 / PREFIX | 6102 / s6102 |
| SEED / VERIFY_SEED | 6102 / 266102 |
| DOMAIN_ID / PORT_BASE | 6 / 8702 |
| Database | s6102_rel |

### Environment

MacBook Air (Mac16,12), Apple M4, 10 CPU cores, 24 GB memory, macOS 15.7.4. The hardware identity comes from the saved host probe. Integration and Part 5 used Python 3.12.14 and MySQL 8.4.11; earlier Parts 2–3 used Python 3.12.13.

Local model: qwen2.5:7b, Q4_K_M, Ollama 0.33.0. Settings: temperature 0, context 4096, reasoning disabled, 60-second model timeout, four returned model responses and at most three tool calls per response.

## Part 1 — Database output



```
+---------------+---------------+---------------+
| student       | database_name | mysql_version |
+---------------+---------------+---------------+
| Pragya Apurva | s6102_rel     | 8.4.11        |
+---------------+---------------+---------------+
+----+----------------------------+---------------+-----------------------------+---------------------+---------------------+
| id | name                       | company       | email                       | created_utc         | updated_utc         |
+----+----------------------------+---------------+-----------------------------+---------------------+---------------------+
|  1 | Integration Campus Manager | Legacy import | legacy-manager-1@example.com | 2026-10-04 03:39 | 2026-10-04 03:39 |
|  2 | Integration Garden Manager | Legacy import | legacy-manager-2@example.com | 2026-10-04 03:39 | 2026-10-04 03:39 |
+----+----------------------------+---------------+-----------------------------+---------------------+---------------------+
+----+----------------------------+--------------+-----------------+------------+---------------------+---------------------+
| id | listing_title              | listing_code | available_units | manager_id | created_utc         | updated_utc         |
+----+----------------------------+--------------+-----------------+------------+---------------------+---------------------+
|  1 | Integration Campus Rental 1 | S6102-R-1    |               1 |          1 | 2026-10-04 03:39 | 2026-10-04 03:39 |
|  2 | Integration Campus Rental 2 | S6102-R-2    |               1 |          1 | 2026-10-04 03:39 | 2026-10-04 03:39 |
+----+----------------------------+--------------+-----------------+------------+---------------------+---------------------+
+-------------------+----------------------------+-----------------+
| TABLE_NAME        | CONSTRAINT_NAME            | CONSTRAINT_TYPE |
+-------------------+----------------------------+-----------------+
| property_managers | uq_property_managers_email | UNIQUE          |
| rentals           | ck_rentals_available_units | CHECK           |
| rentals           | fk_rentals_manager         | FOREIGN KEY     |
| rentals           | uq_rentals_listing_code    | UNIQUE          |
+-------------------+----------------------------+-----------------+
+------------+-----------------------+-------------+
| TABLE_NAME | REFERENCED_TABLE_NAME | DELETE_RULE |
+------------+-----------------------+-------------+
| rentals    | property_managers     | RESTRICT    |
+------------+-----------------------+-------------+
(base) pragyaapurva@Pragyas-MacBook-Air ~ %
```

**Figure 1. s6102_rel: linked records, unique/check constraints and the RESTRICT rule.**

## Part 1 — Database and validation

One property manager can have many rentals. A rental may be unassigned or reference one manager. The foreign key enforces this relationship; RESTRICT prevents deleting a manager whose rentals still exist.

| Table | Fields and rules |
|---|---|
| property_managers | Primary-key id; name; company; unique email; created_at and updated_at in UTC. |
| rentals | Primary-key id; unique listing_code; available_units ≥ 0, default 1; nullable manager_id foreign key; UTC timestamps; existing title, address, type, description and submitter fields. |
| users / sessions | Existing relationships and password hashes preserved. Passwords are stored as hashes, not plain text. |

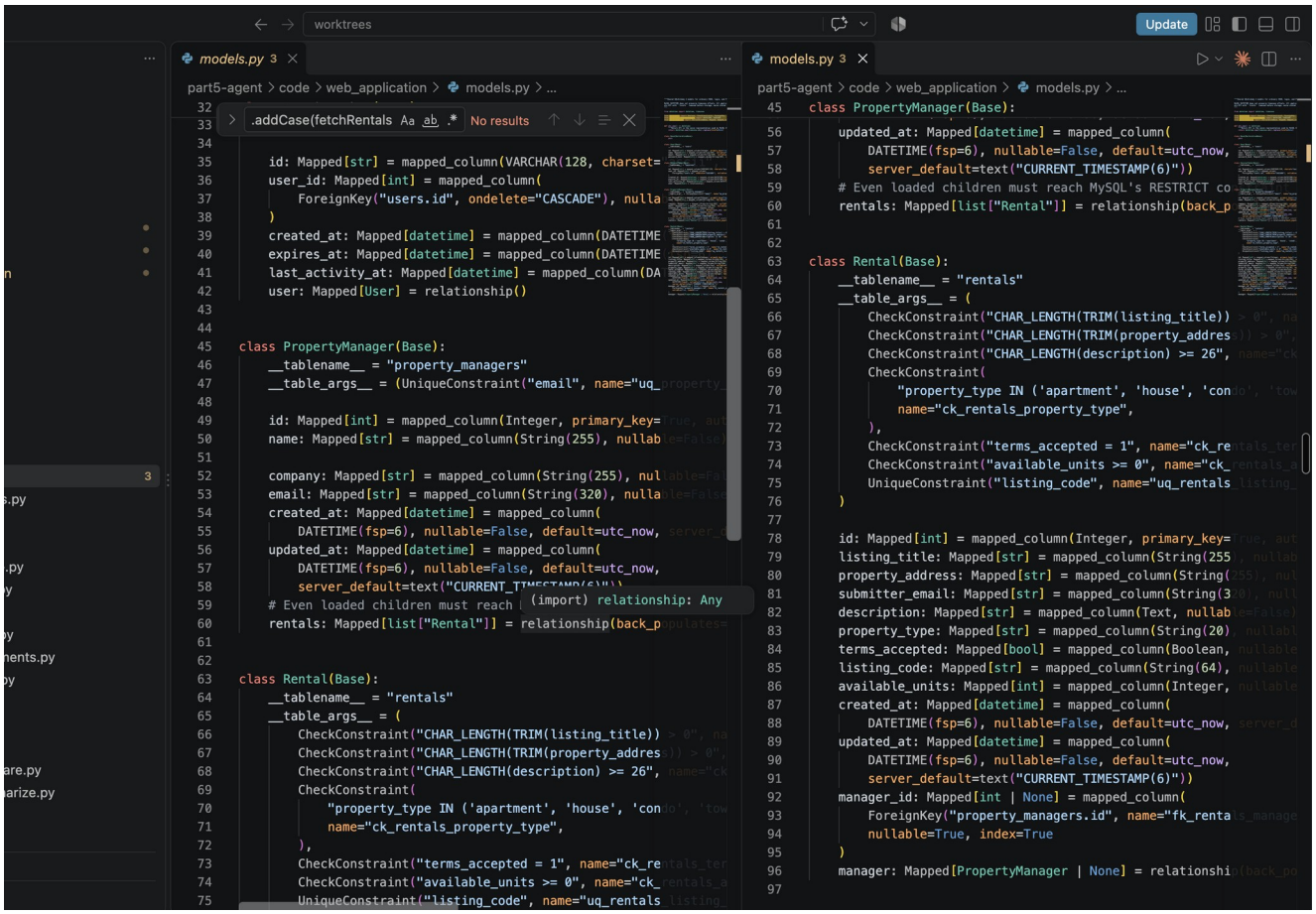



**Figure 16. VS Code: PropertyManager and Rental definitions in code/web_application/models.py.**

Migration 004 backfills the new fields, validates existing records, then enforces constraints. It preserves IDs, relationships, password hashes and sessions. Legacy timestamps use the persisted upgrade time because the original times are unknown. Eleven populated-migration tests passed, including repeat execution.

### API rules and observed errors

Observed responses: 422 for invalid fields, negative units or malformed codes; 409 for duplicate email/code or deleting a referenced manager; 404 for missing records.

Pydantic validates types, required fields and formats. Codes follow S6102-R-<positive integer>; omission on create generates a code. Both list APIs use limit (1–100, default 50), offset ≥ 0, stable ID order and X-Total-Count for the full matching total.

## Part 1 — API implementation

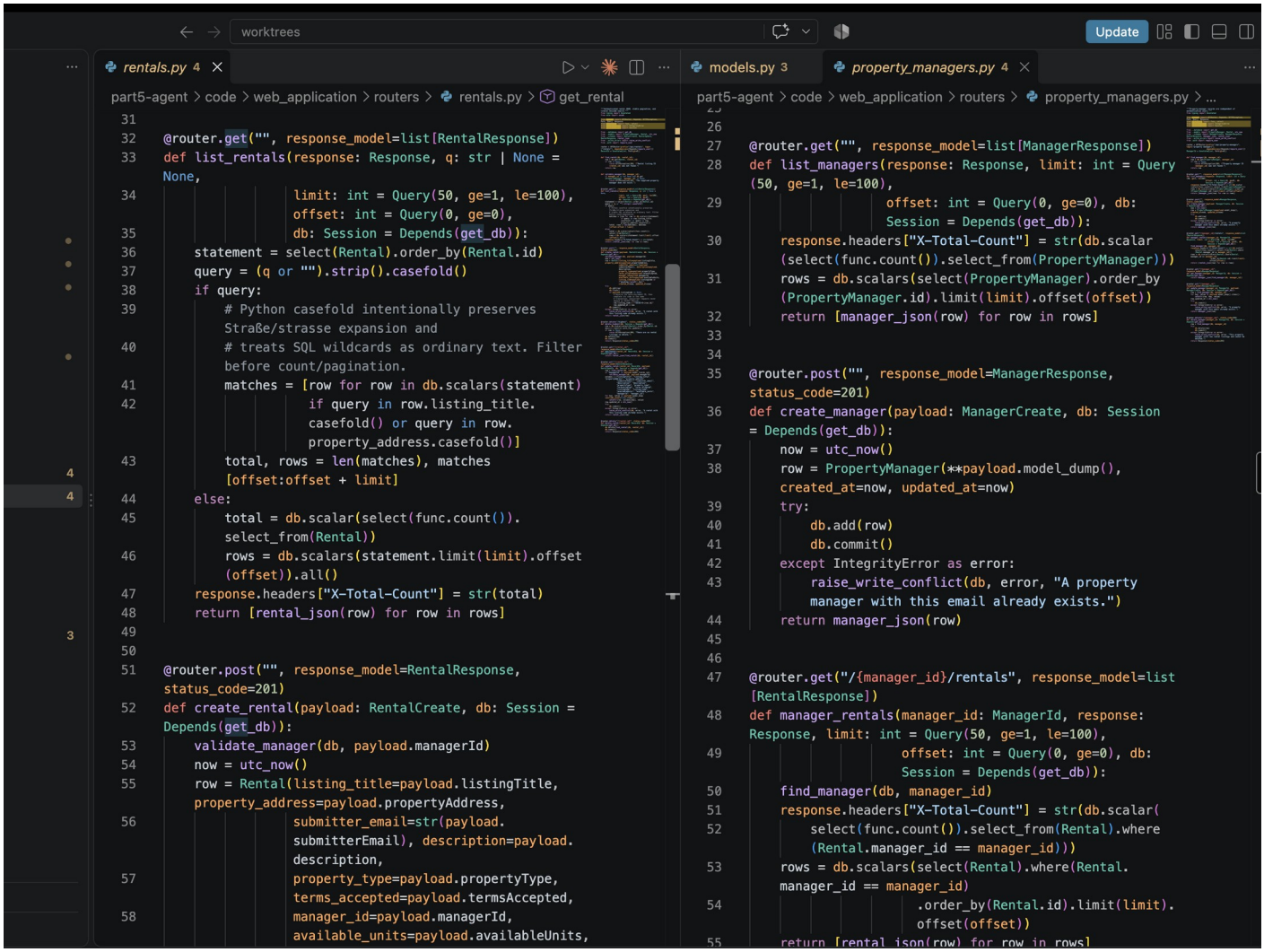



**Figure 17. VS Code: rental list/search and create excerpt (left); manager list, create and relationship query (right). Files: code/web_application/routers/rentals.py and property_managers.py.**

The list handlers apply pagination and return X-Total-Count. The relationship query filters by manager_id and orders matching rentals by ID. The following Postman captures show the API outcomes for both entities.

## Part 1 — Manager create and reads

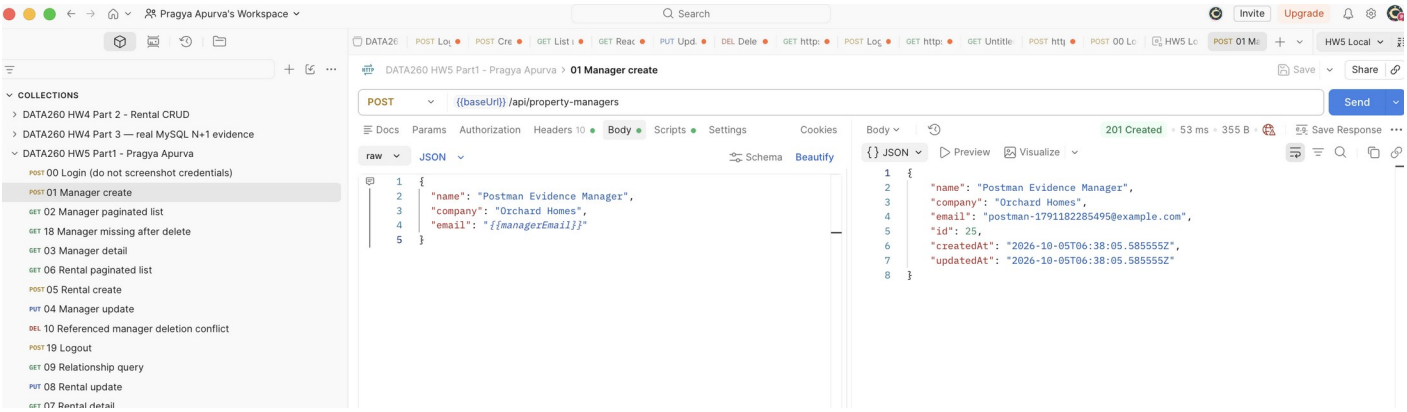



**Figure 2a. POST /api/property-managers: manager 25 created (201).**

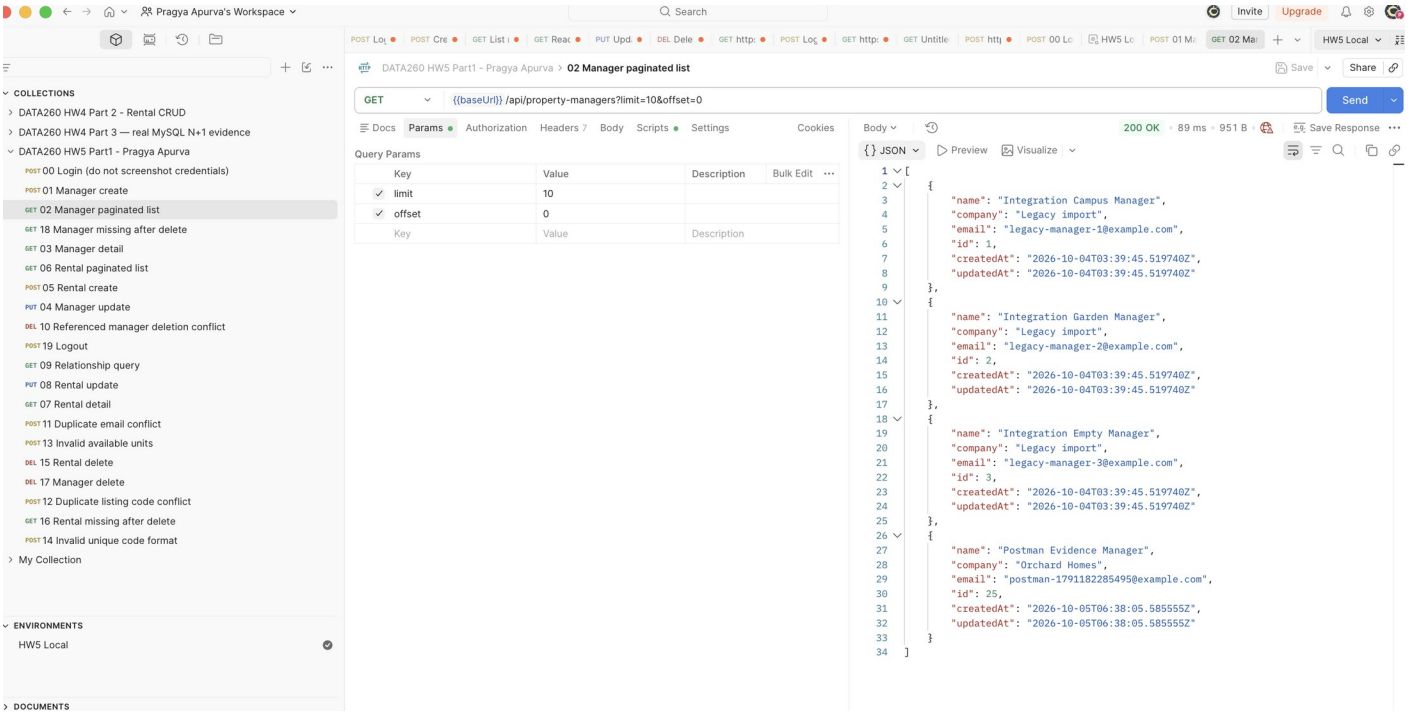



**Figure 2b. Paginated manager list (200); limit=10, offset=0.**

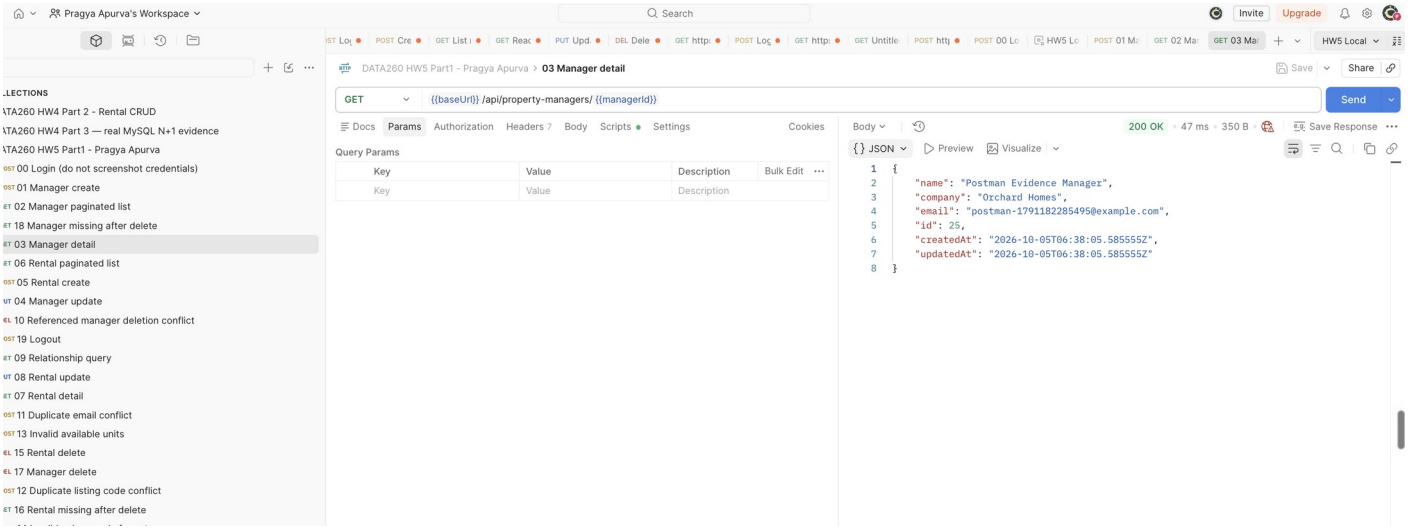



**Figure 2c. GET /api/property-managers/25 returns the created manager (200).**

## Part 1 — Manager update and delete

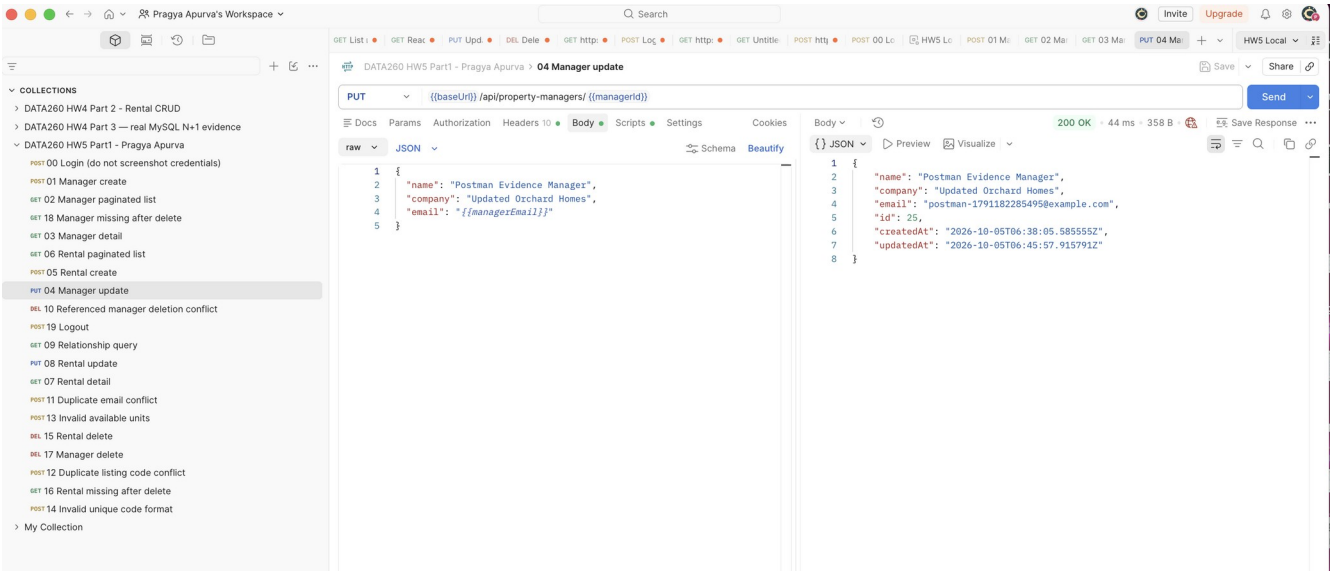



**Figure 2d. PUT manager 25 changes company to Updated Orchard Homes (200).**

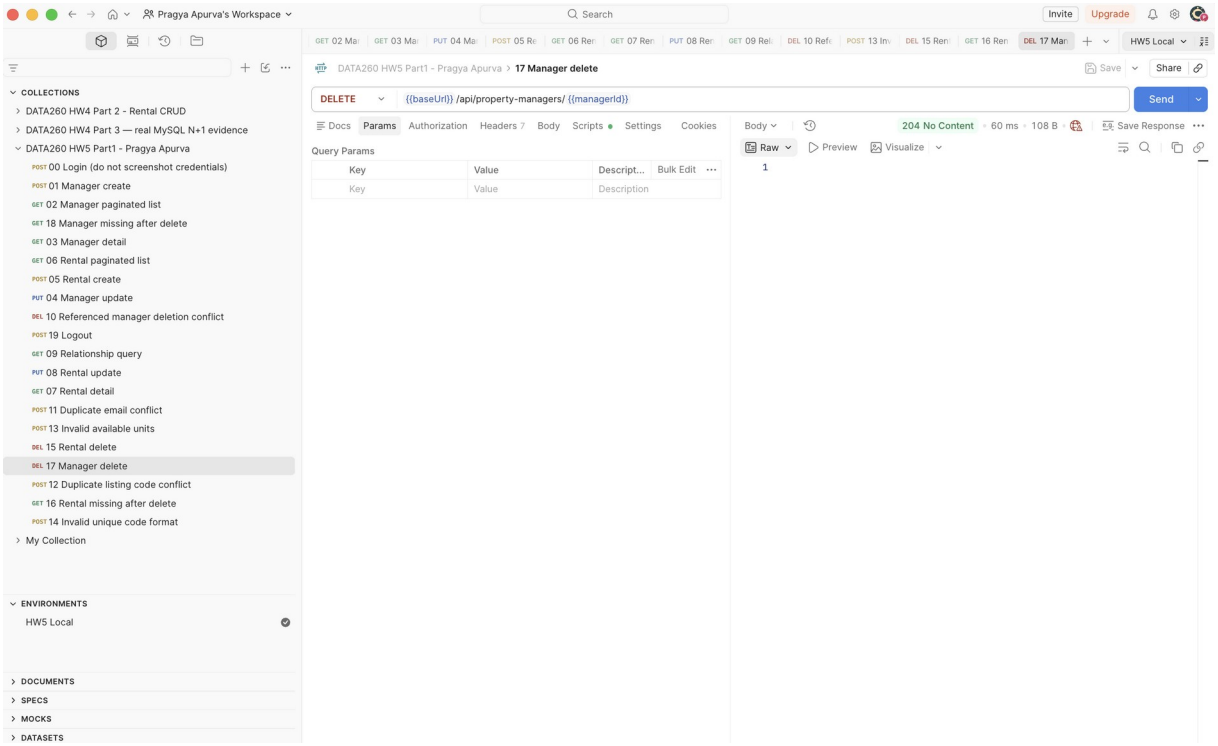



**Figure 2e. DELETE manager 25 succeeds after rental removal (204, empty body).**

CRUD means create, read, update and delete. Calls require the existing login session. Failed writes roll back; manager deletion follows rental removal. The list total was four.

## Part 1 — Rental create and list

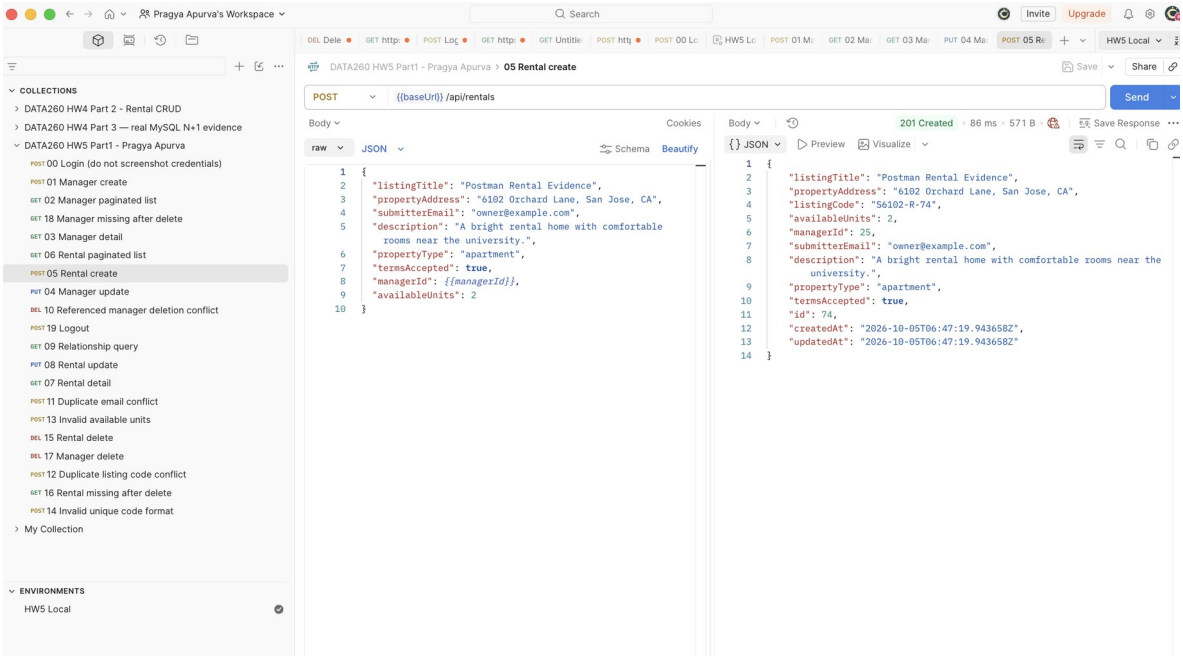



**Figure 3a. POST /api/rentals creates rental 74 linked to manager 25 (201).**

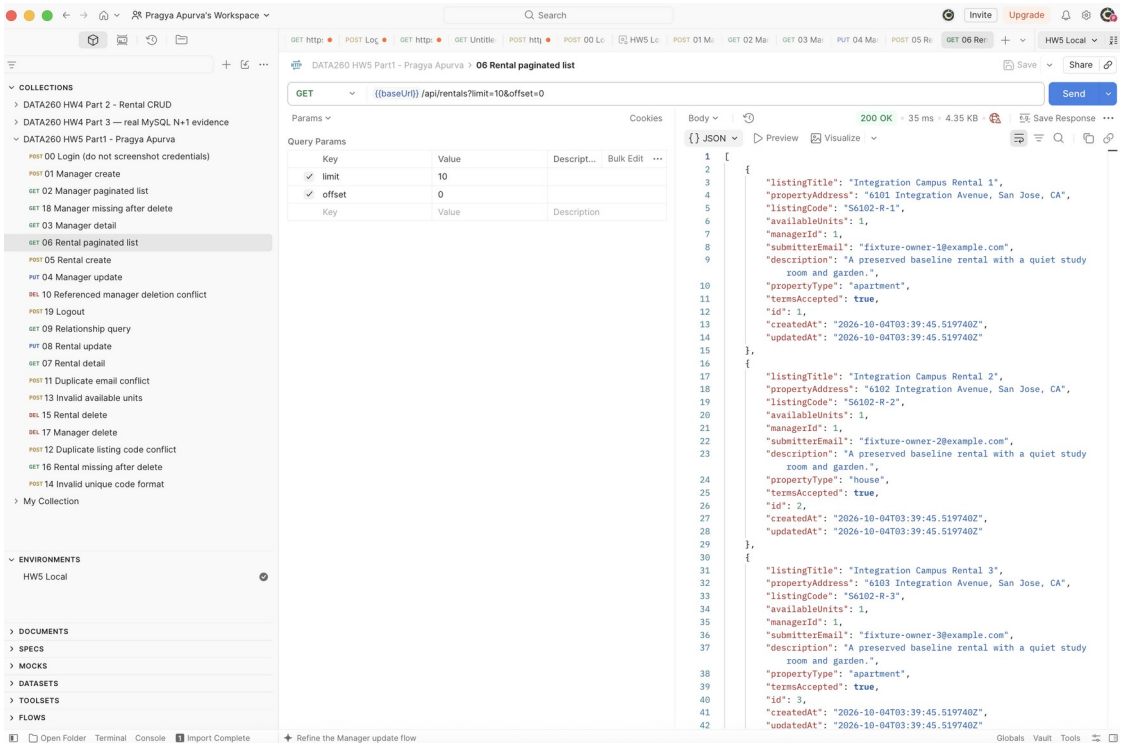



**Figure 3b. Paginated rental list (200); X-Total-Count was 13 for this capture.**

## Part 1 — Rental detail and update

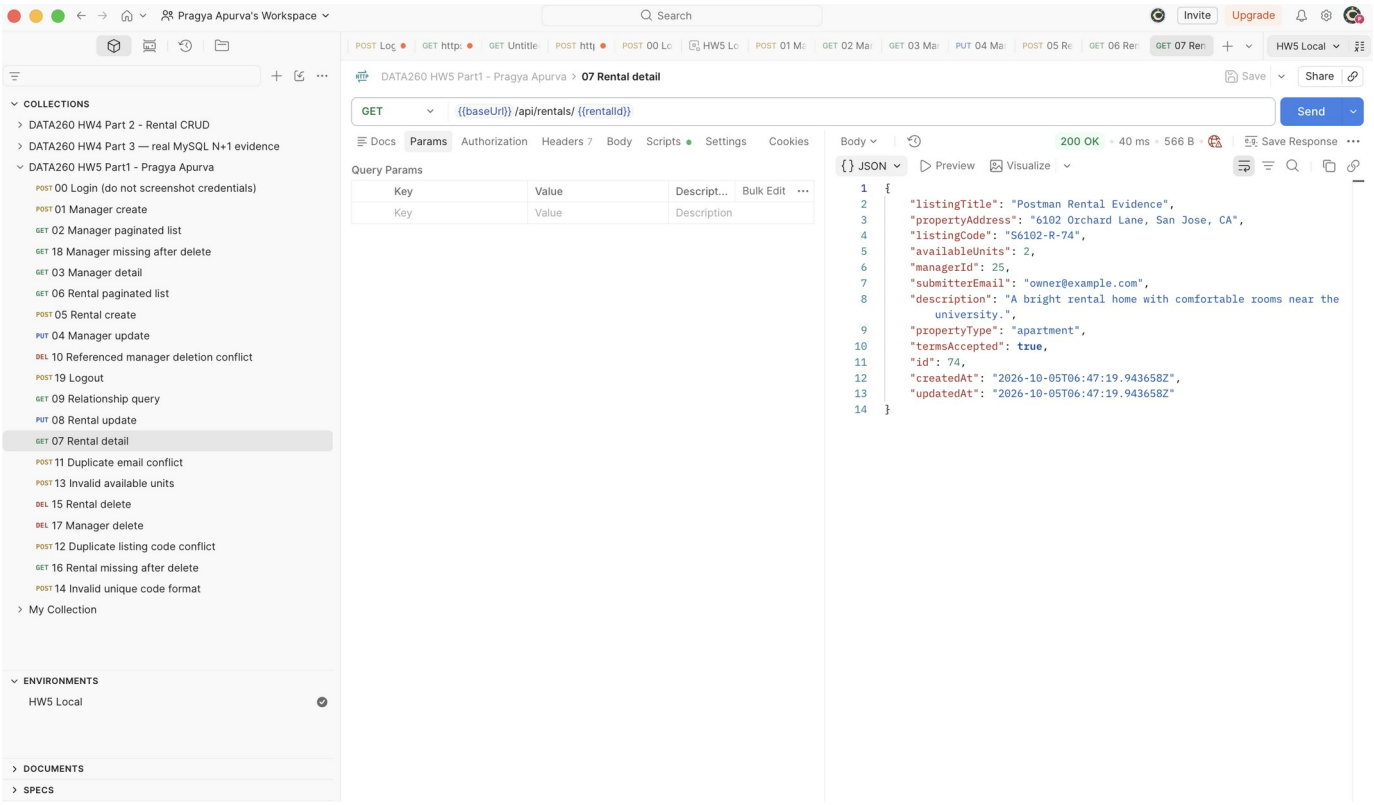



**Figure 3c. GET rental 74 returns its details and generated code S6102-R-74 (200).**

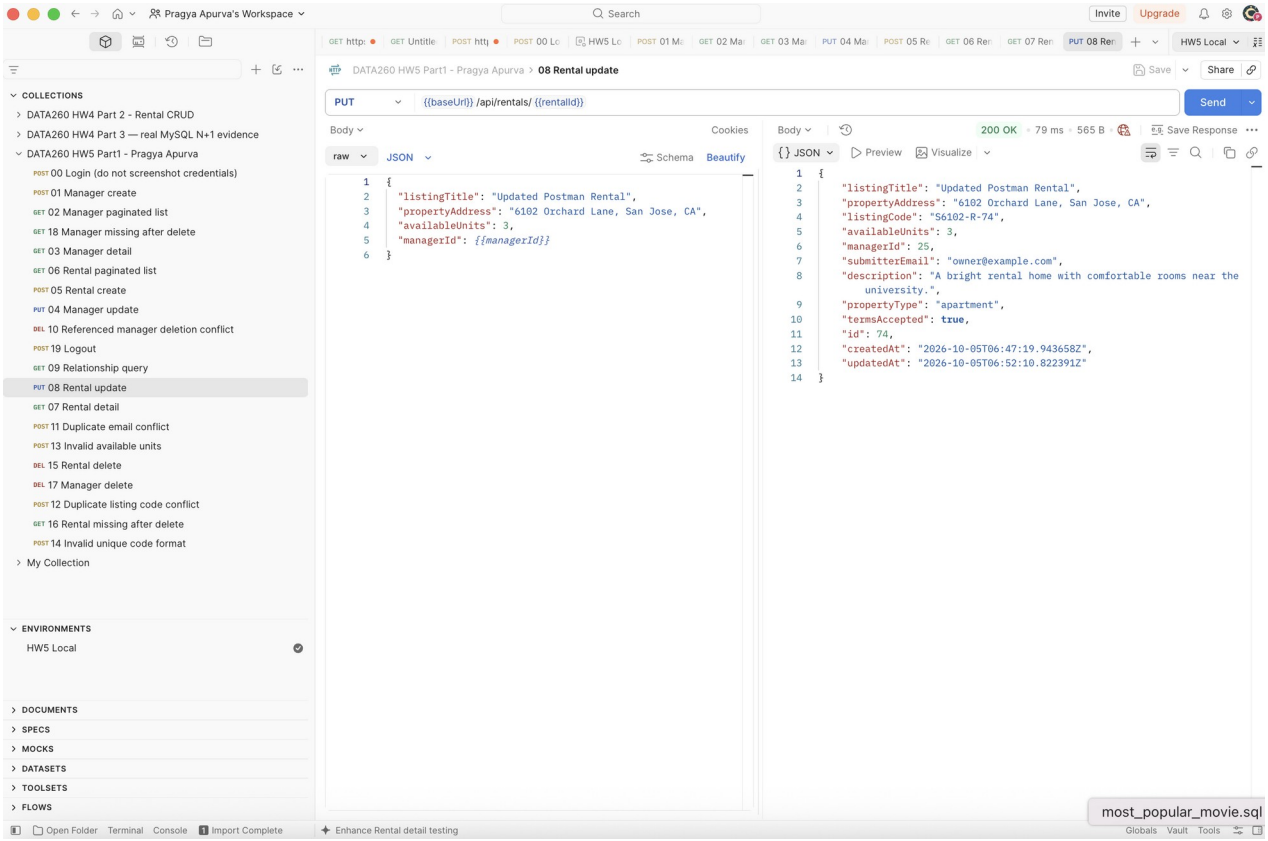



**Figure 3d. PUT rental 74 updates the title and available units to 3 (200).**

## Part 1 — Rental delete and relationship

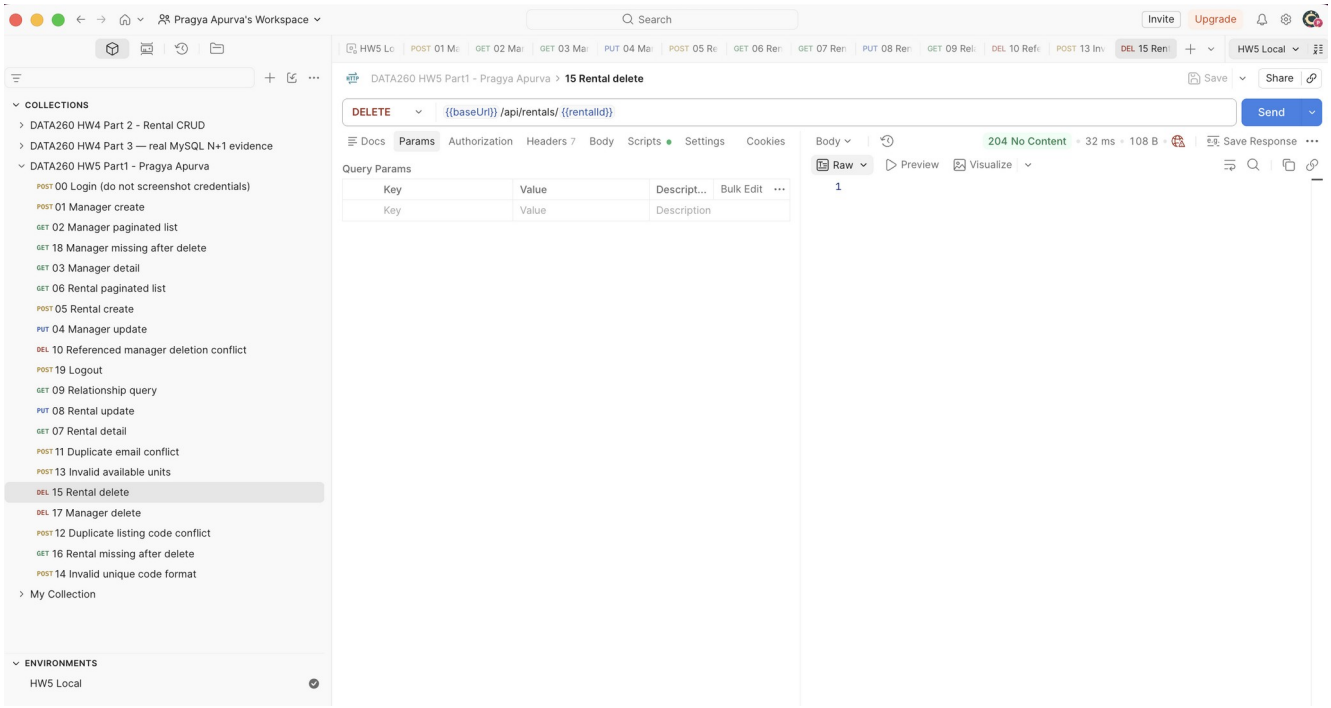



**Figure 3e. DELETE rental 74 returns 204 with no response body.**

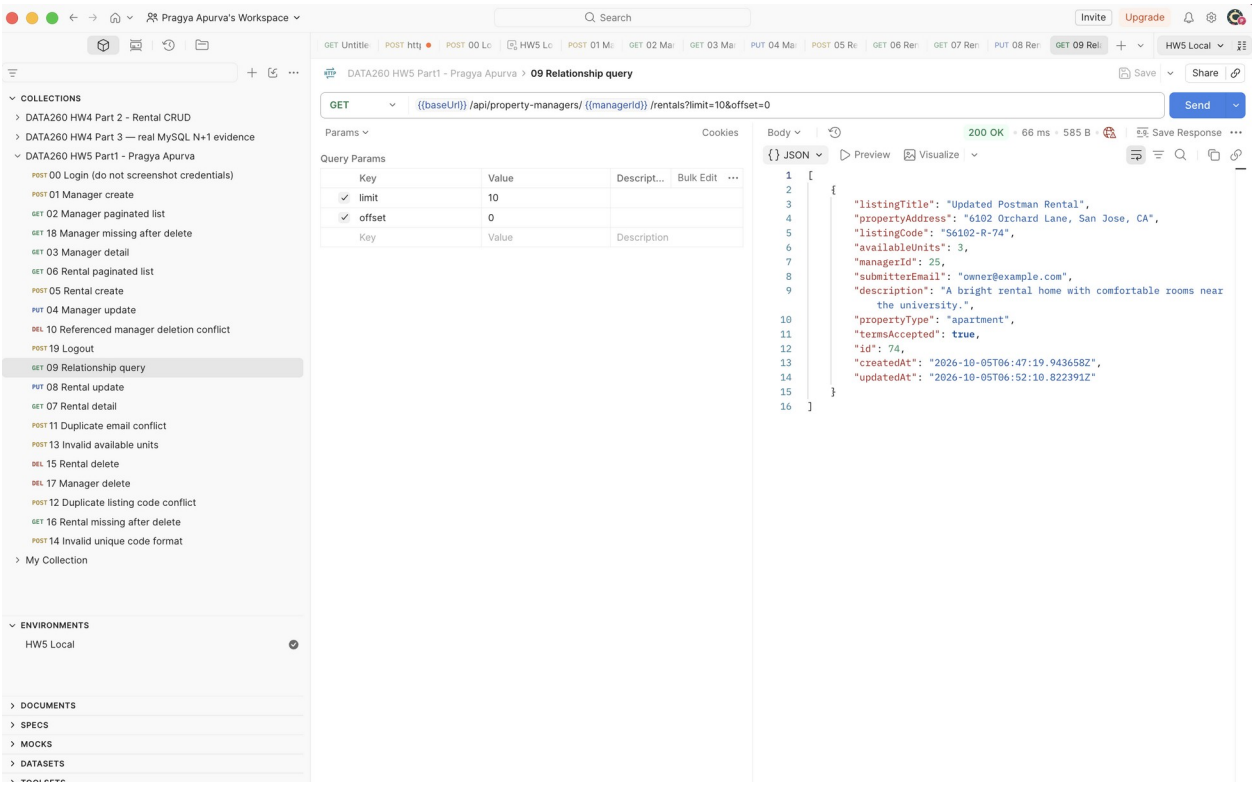



**Figure 4. Before deletion, GET /api/property-managers/25/rentals returns rental 74.**

The relationship endpoint returned one matching rental and X-Total-Count: 1. Deleting that rental removed the reference, allowing the later manager deletion. A read after rental deletion returned 404.

## Part 1 — Redux Home and Create

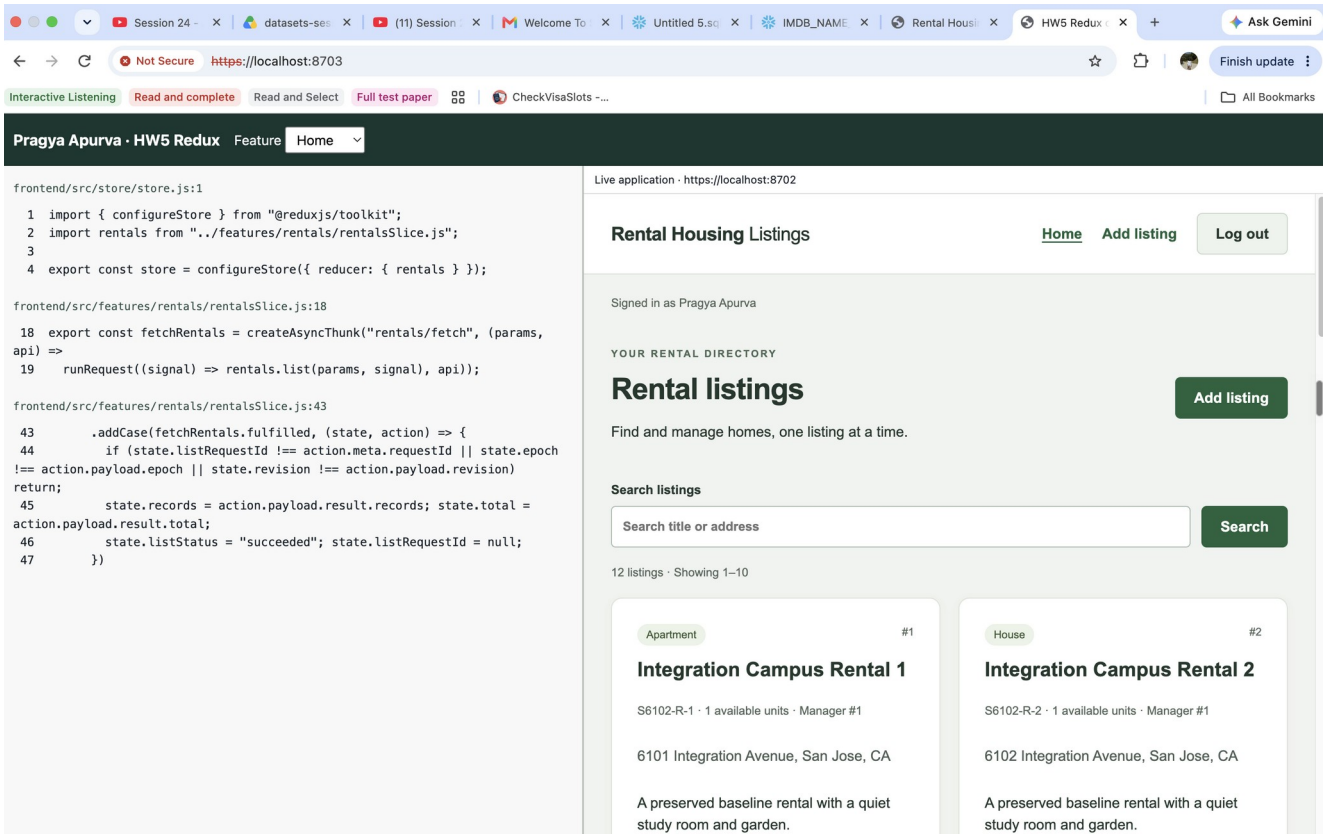



**Figure 6a. Home: store, fetch thunk and reducer beside the live list (12 listings).**

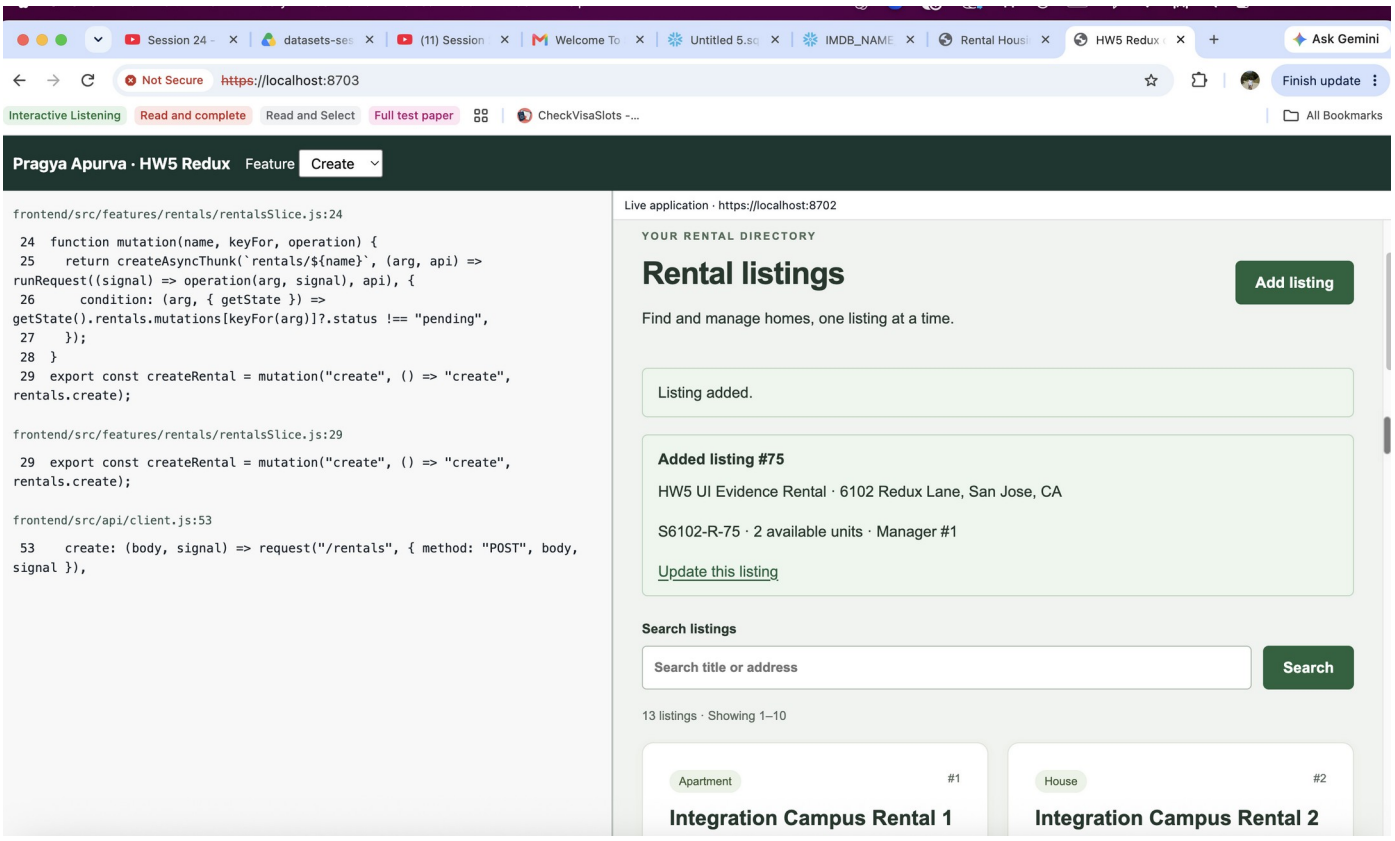



**Figure 6b. Create: thunk and POST code beside new listing 75 with 2 available units.**

## Part 1 — Redux Update and Delete

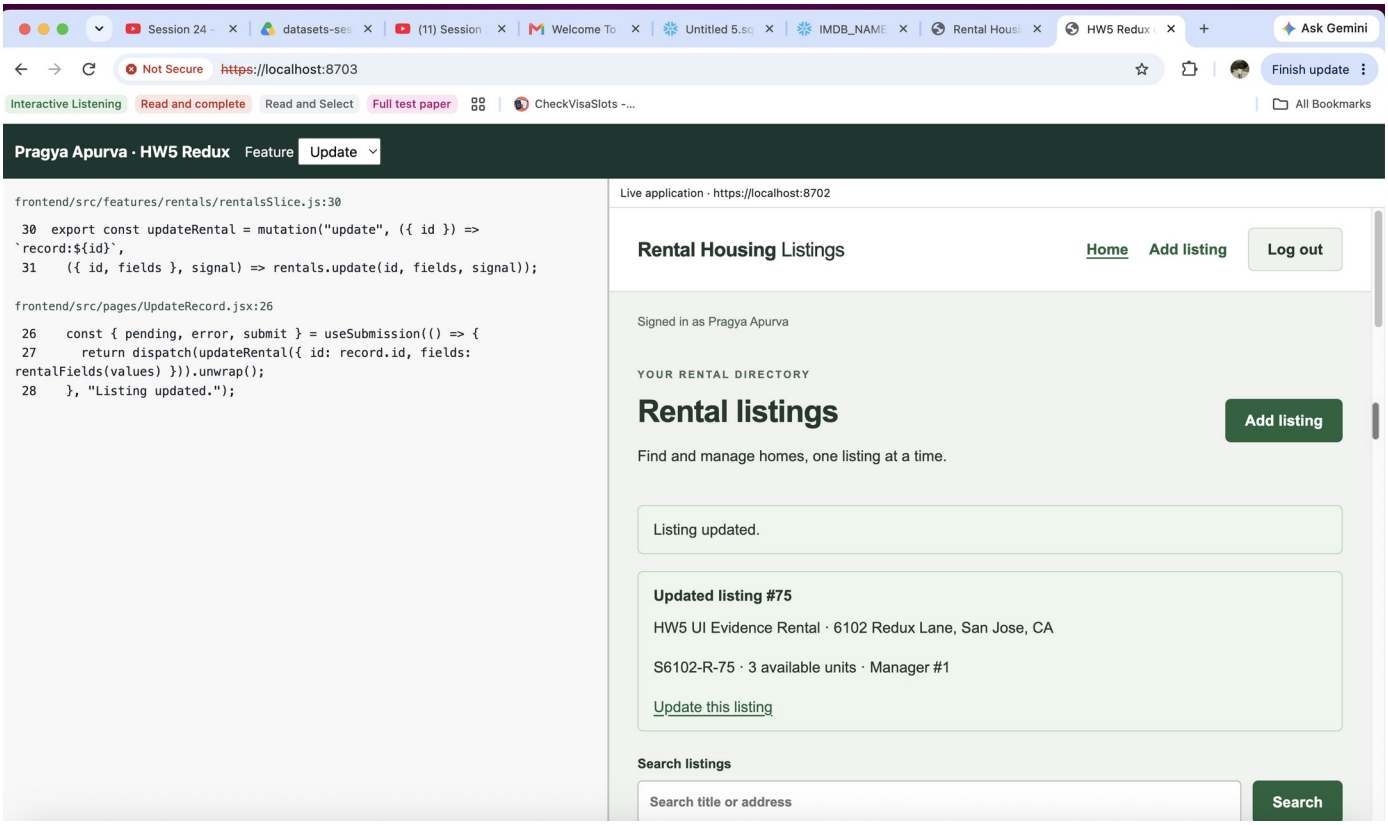



**Figure 7a. Update: dispatched thunk beside listing 75 after changing units to 3.**

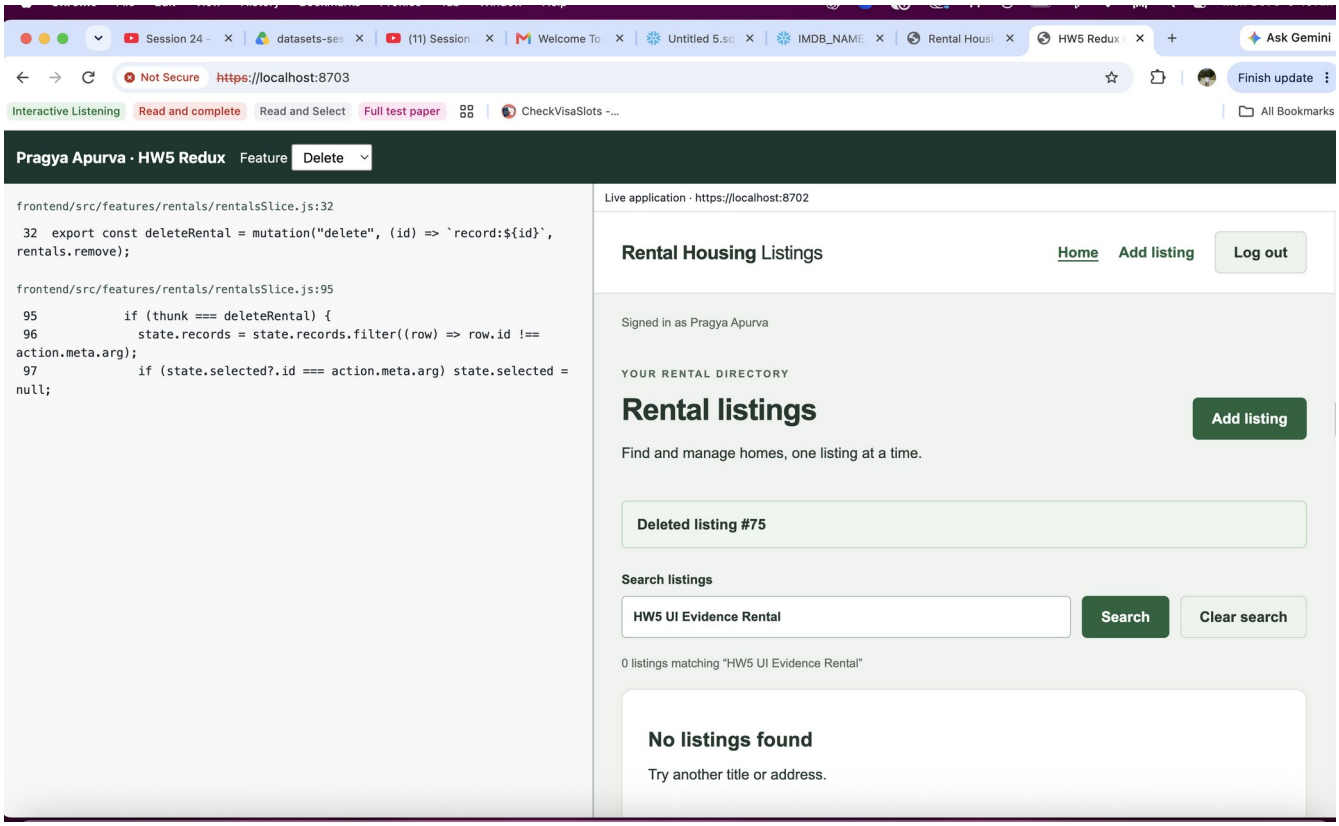



**Figure 7b. Delete: reducer removes listing 75; the live search then returns no listings.**

Redux Toolkit thunks call FastAPI through Axios. Pending/fulfilled/rejected state, request IDs and session/mutation guards reject stale updates and duplicate writes; successful writes refresh the list. Viewer 8703 embeds the live app on 8702.

## Part 2 — MCP tool design

MCP lets a client discover named tools and call them with structured input. Both servers use STDIO: protocol messages go to standard output and diagnostics to standard error. The Meals server makes genuine TheMealDB HTTP requests with a 10-second timeout.

### Part 2A — Meals tools

```
    @server.tool(structured_output=False, annotations=read_only)
    async def search_meals_by_name(query: TextInput, limit: SearchLimit = 5):
        """Search TheMealDB by meal name; return id, name, area, category, and thumb."""
        return await call(upstream.search_meals_by_name(query, limit))

    @server.tool(structured_output=False, annotations=read_only)
    async def meals_by_ingredient(ingredient: TextInput, limit: IngredientLimit = 12):
        """Find meals by one main ingredient; return id, name, and thumb."""
        return await call(upstream.meals_by_ingredient(ingredient, limit))

    @server.tool(structured_output=False, annotations=read_only)
    async def random_meal():
        """Return one random full recipe, including its ingredient measurements."""
        return await call(upstream.random_meal())

    @server.tool(structured_output=False, annotations=read_only)
    async def meal_details(id: MealId):
        """Look up a positive meal ID and return the full recipe."""
        return await call(upstream.meal_details(id))

    return server


def main():
    create_server().run(transport="stdio")


if __name__ == "__main__":
    main()
```

**Figure 18. VS Code: four Meals tools and STDIO startup in src/mcp_servers/meals.py.**

Each function is registered with @server.tool(structured_output=False, annotations=read_only); create_server().run(transport="stdio") starts the server.

| Tool / upstream endpoint | Input and output |
|---|---|
| search_meals_by_name / search.php?s= | Nonempty query; limit 1–25 (default 5). Returns id, name, area, category and thumbnail. |
| meals_by_ingredient / filter.php?i= | Nonempty ingredient; positive limit (default 12). Returns id, name and thumbnail. |
| random_meal / random.php | No inputs. Returns one full recipe and ingredient measurements. |
| meal_details / lookup.php?i= | Positive integer or digit-string id. Returns recipe instructions, ingredients and links. |

Invalid inputs are rejected; empty lookups report no matches. Upstream failures become MCP tool errors instead of fabricated recipes. Figures 8–9 show one real call per tool.

## Part 2A — Meals search and ingredient

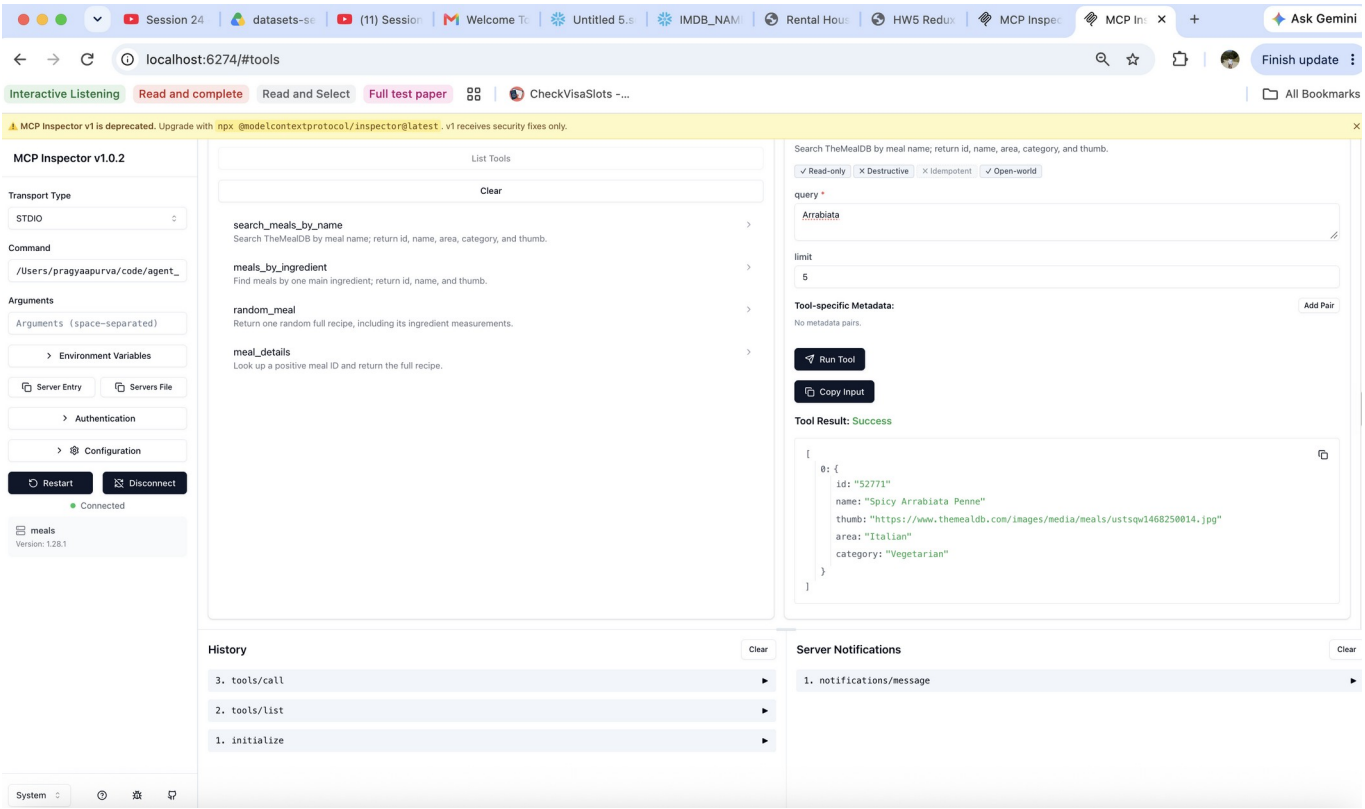



**Figure 8a. query=Arrabiata, limit=5 → Spicy Arrabiata Penne (52771), Italian/Vegetarian.**

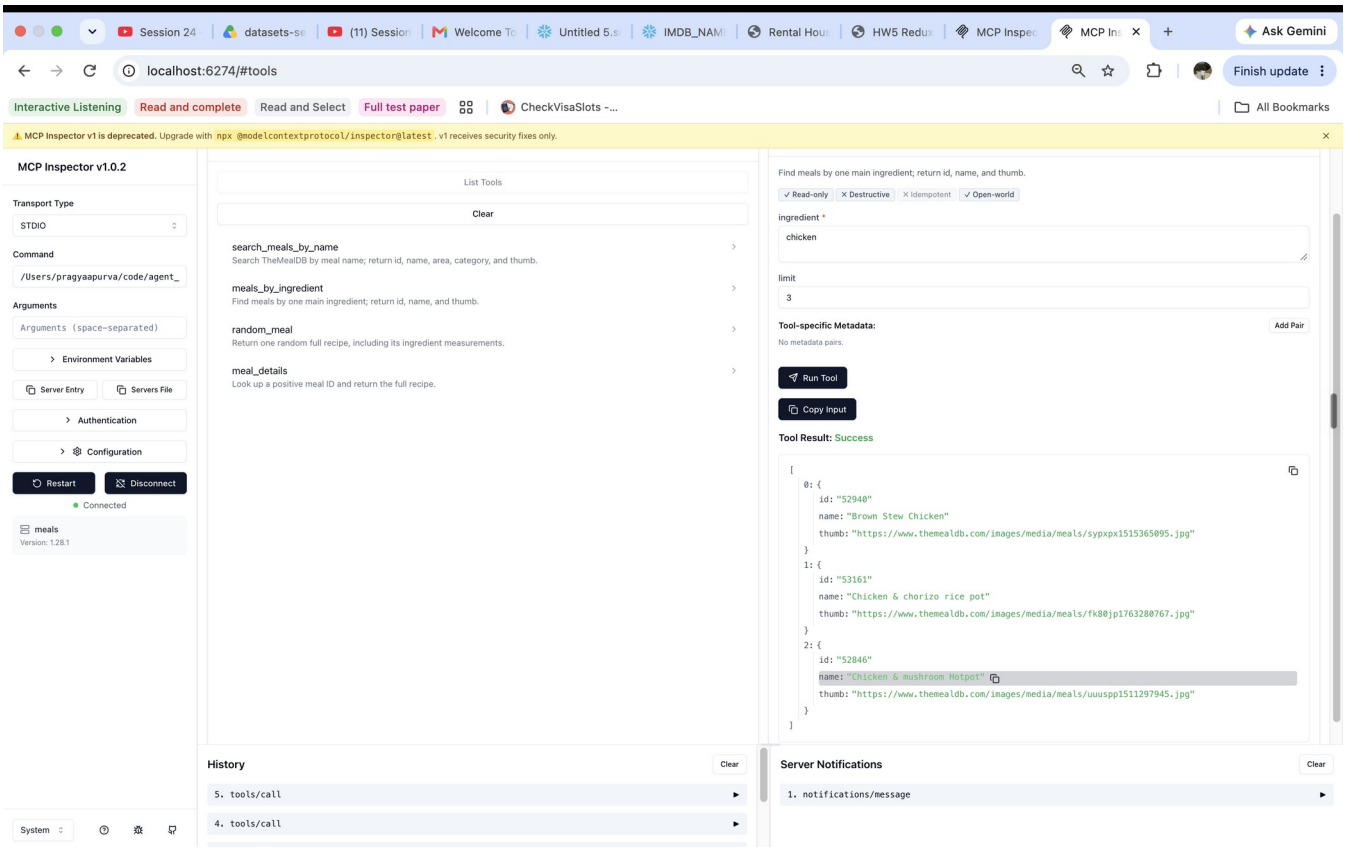



**Figure 8b. ingredient=chicken, limit=3 → IDs 52940, 53161 and 52846.**

## Part 2A — Random meal and detail

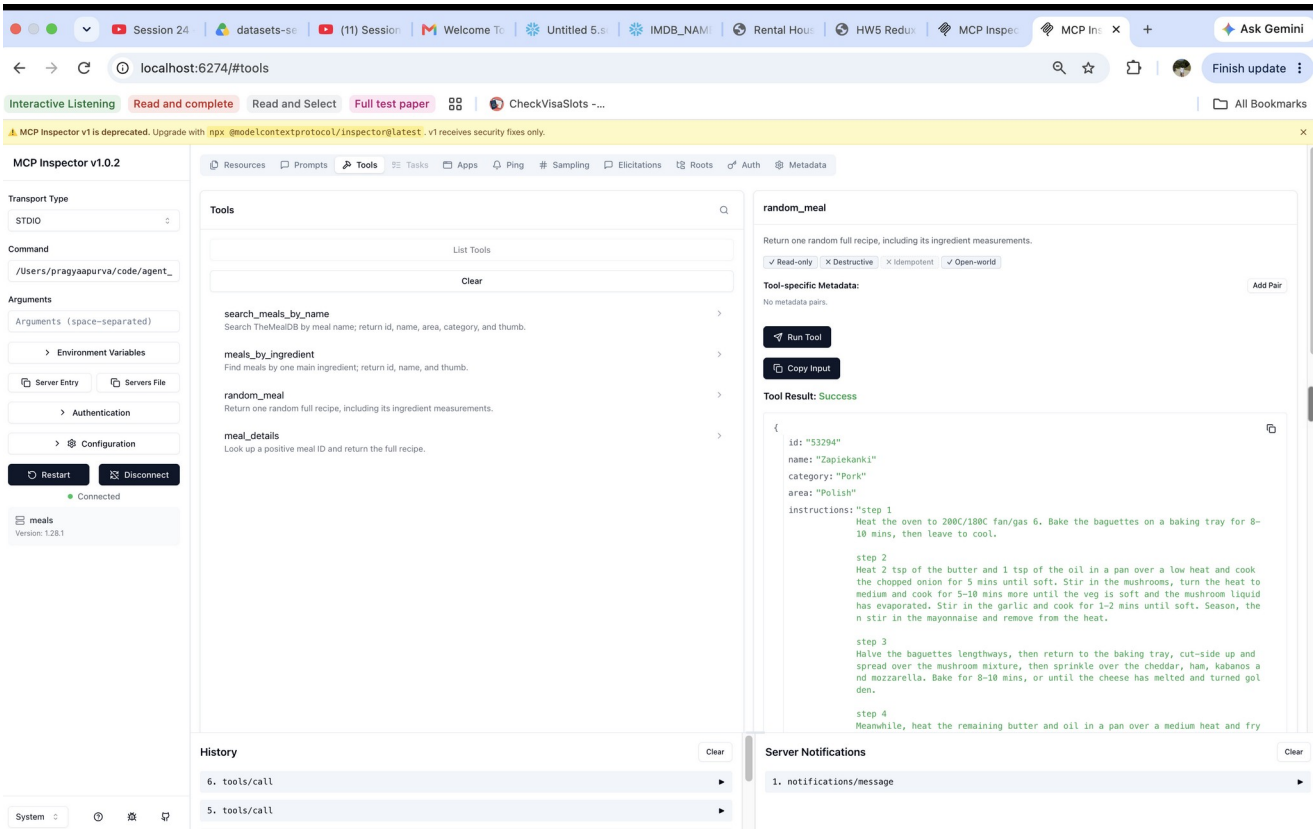



**Figure 9a. random_meal({}) → Zapiekanki (53294), Polish/Pork, with recipe instructions.**

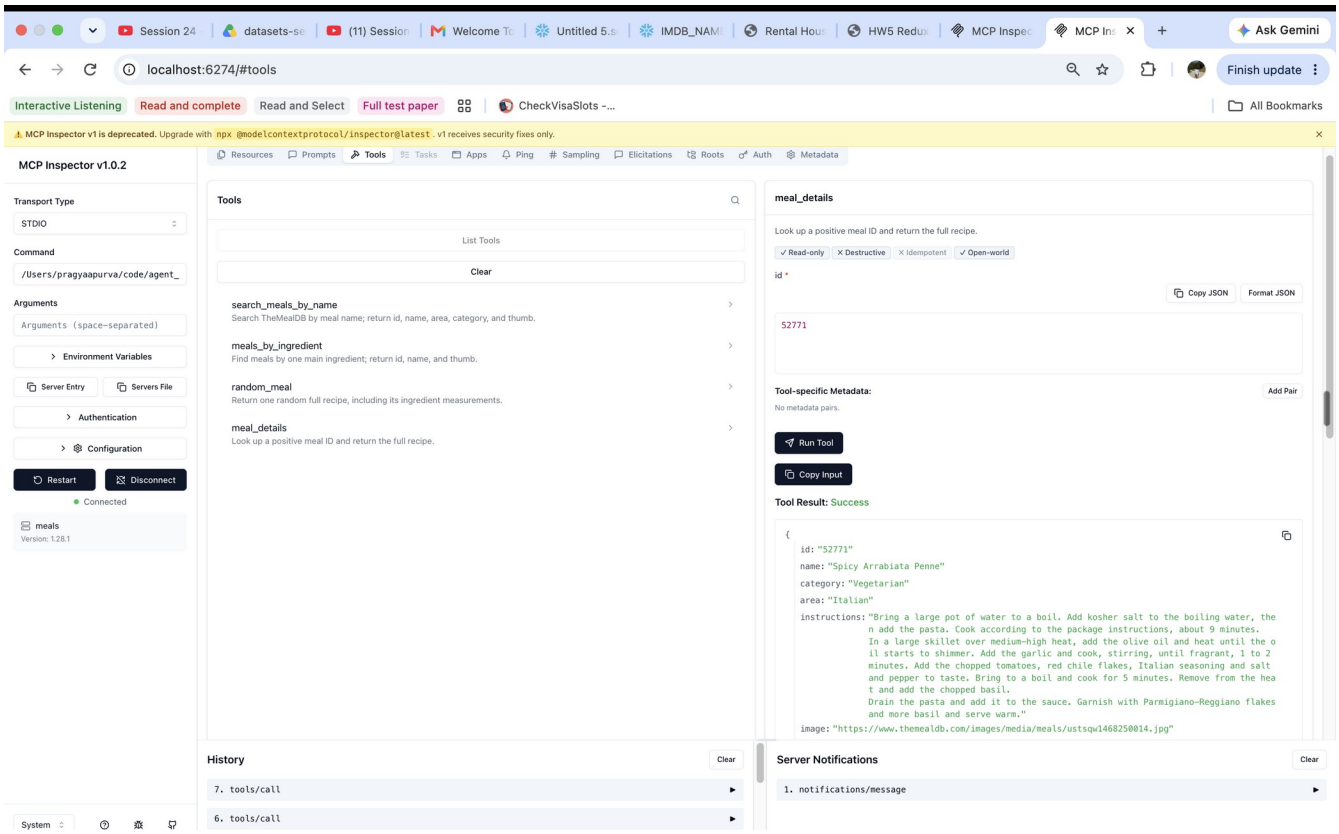
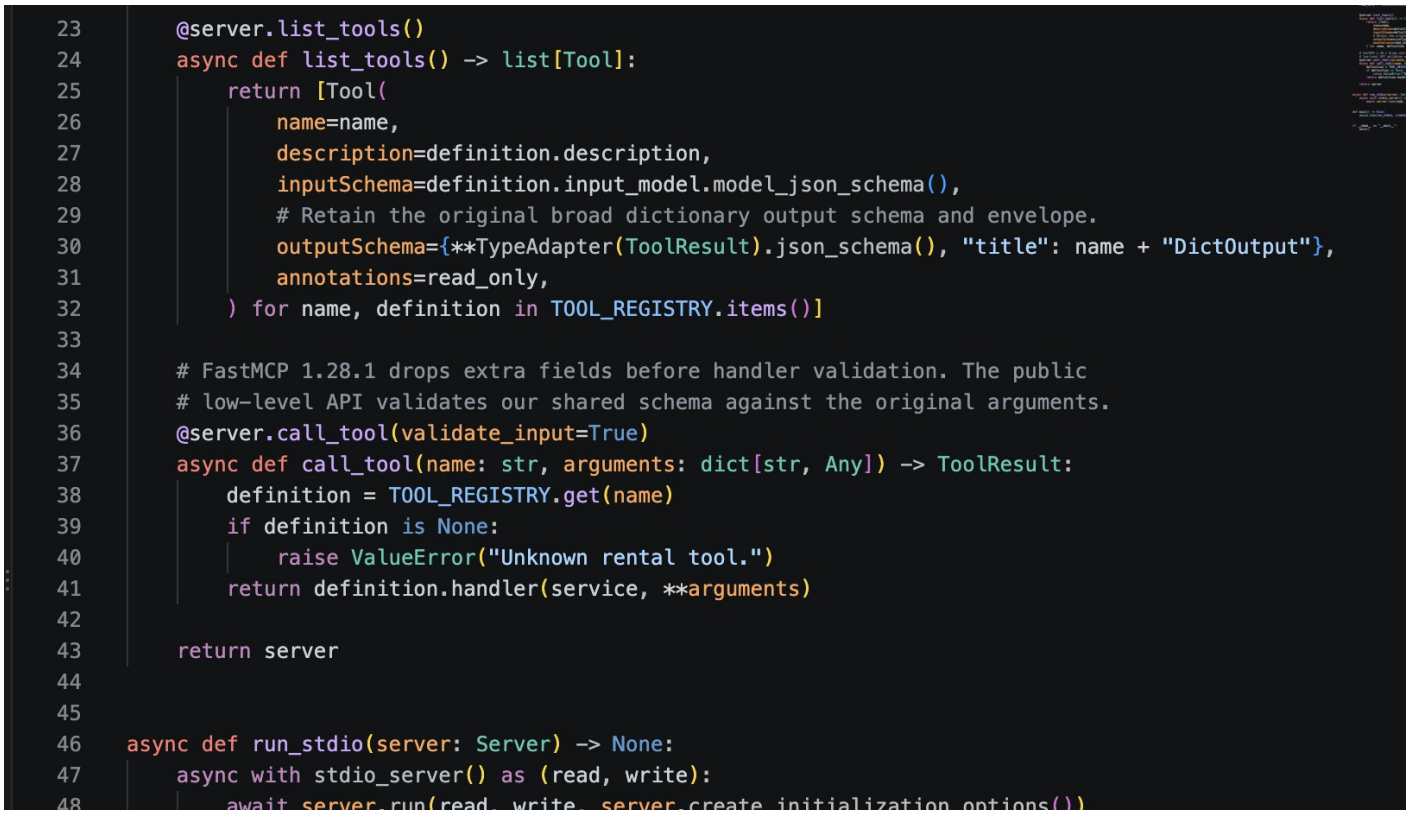



**Figure 9b. meal_details({"id":52771}) → Spicy Arrabiata Penne recipe.**

## Part 2B — Rental tool implementation

The domain server exposes exactly search_rentals, get_rental and count_rentals. Public fields exclude submitter email. Every application result has {ok, data, error}: success has ok=true/error=null; failure has ok=false/data=null.



```python
    @server.list_tools()
    async def list_tools() -> list[Tool]:
        return [Tool(
            name=name,
            description=definition.description,
            inputSchema=definition.input_model.model_json_schema(),
            # Retain the original broad dictionary output schema and envelope.
            outputSchema={**TypeAdapter(ToolResult).json_schema(), "title": name + "DictOutput"},
            annotations=read_only,
        ) for name, definition in TOOL_REGISTRY.items()]

    # FastMCP 1.28.1 drops extra fields before handler validation. The public
    # low-level API validates our shared schema against the original arguments.
    @server.call_tool(validate_input=True)
    async def call_tool(name: str, arguments: dict[str, Any]) -> ToolResult:
        definition = TOOL_REGISTRY.get(name)
        if definition is None:
            raise ValueError("Unknown rental tool.")
        return definition.handler(service, **arguments)

    return server


async def run_stdio(server: Server) -> None:
    async with stdio_server() as (read, write):
        await server.run(read, write, server.create_initialization_options())
```

**Figure 19a. VS Code: tool discovery, schema registration and validated dispatch in src/mcp_servers/rentals.py.**



```python


def success(data: dict[str, Any]) -> ToolResult:
    return {"ok": True, "data": data, "error": None}


def failure(message: str) -> ToolResult:
    return {"ok": False, "data": None, "error": message}

```

**Figure 19b. VS Code: success and failure envelopes in src/rental_tools/contracts.py.**

Figures 10–11 use the retained seven-rental, three-manager fixture on port 3308 through current code. The Inspector “Success” badge means a valid tool response was delivered; ok=false inside the envelope still reports a rejected operation.

## Part 2B — Rental search

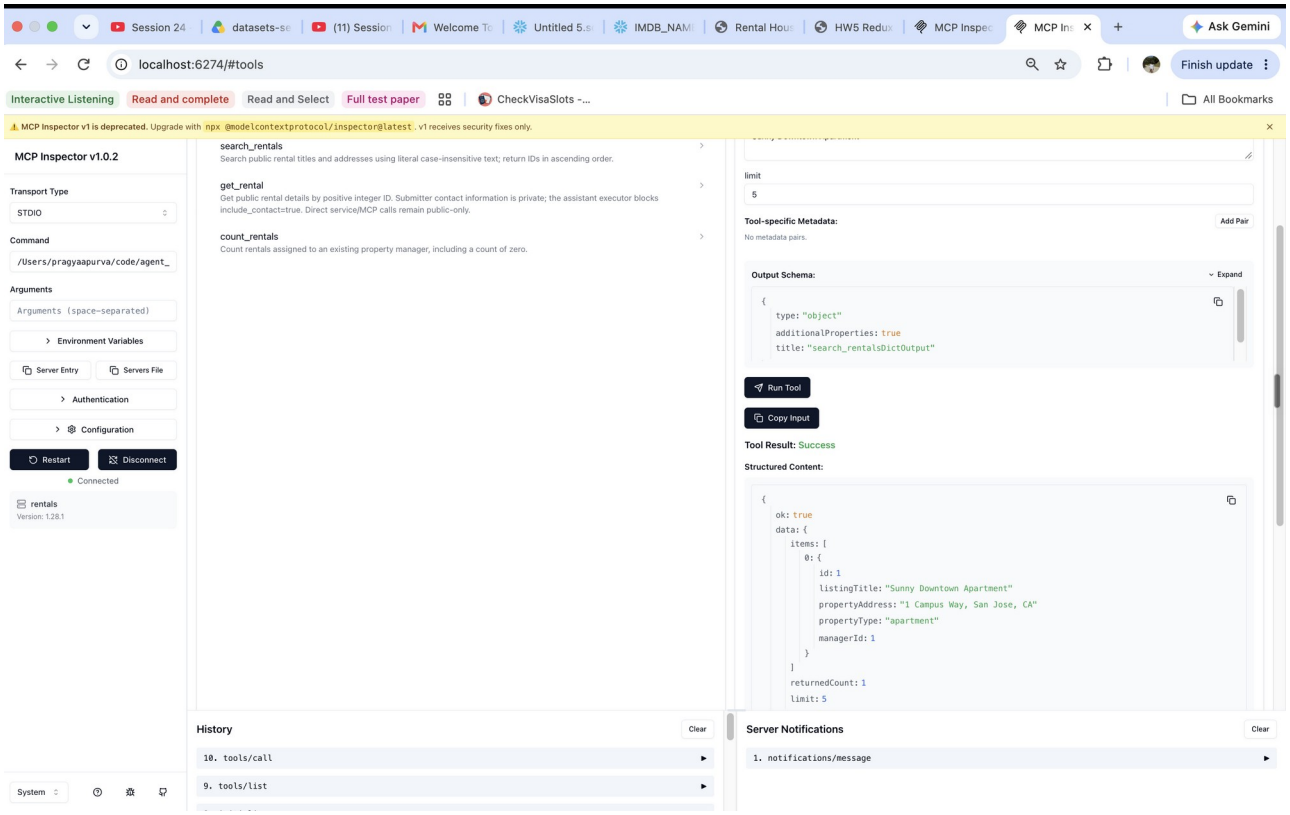



**Figure 10a. Search for Sunny Downtown Apartment, limit=5: one result, rental 1.**

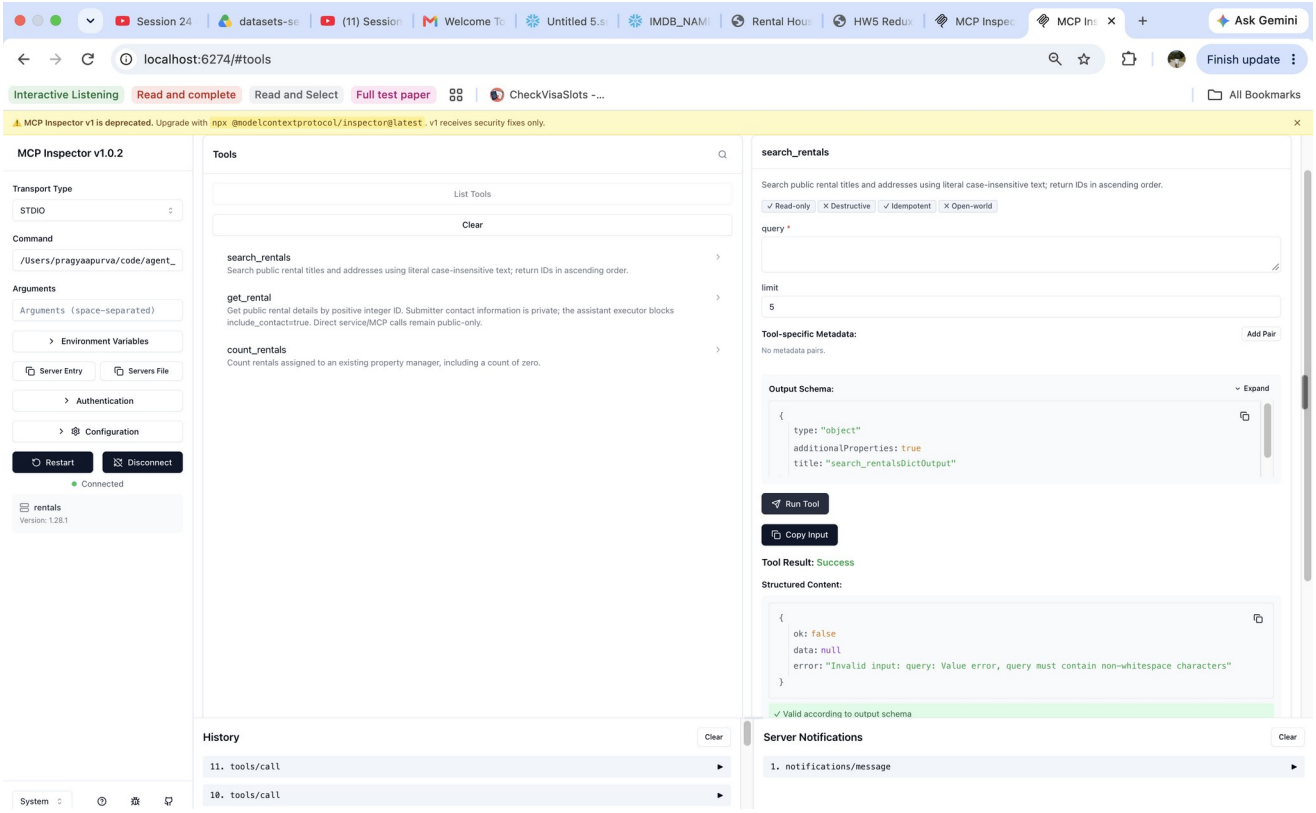



**Figure 10b. {"query":"  ","limit":5}: ok=false; query must contain non-whitespace characters.**

## Part 2B — Rental detail

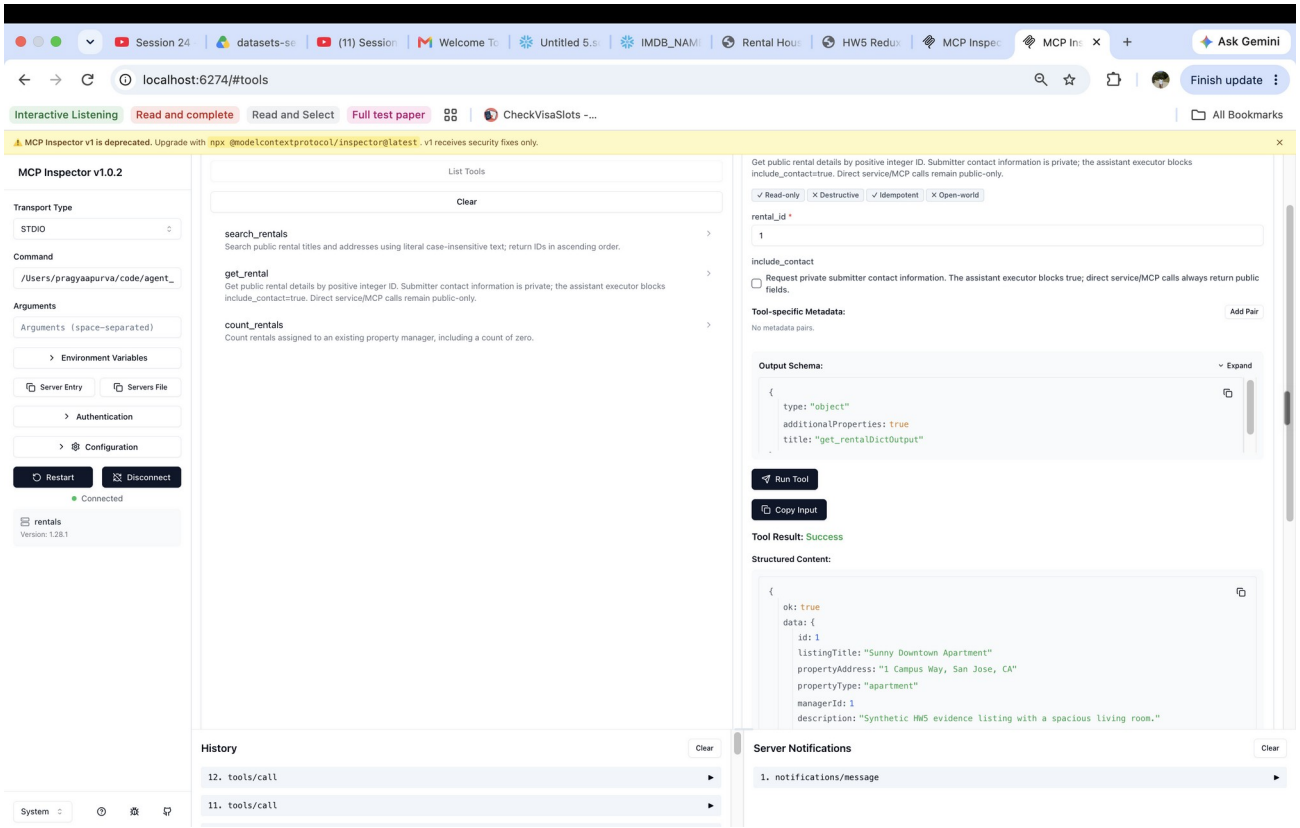



**Figure 11a. {"rental_id":1,"include_contact":false}: public rental details returned.**

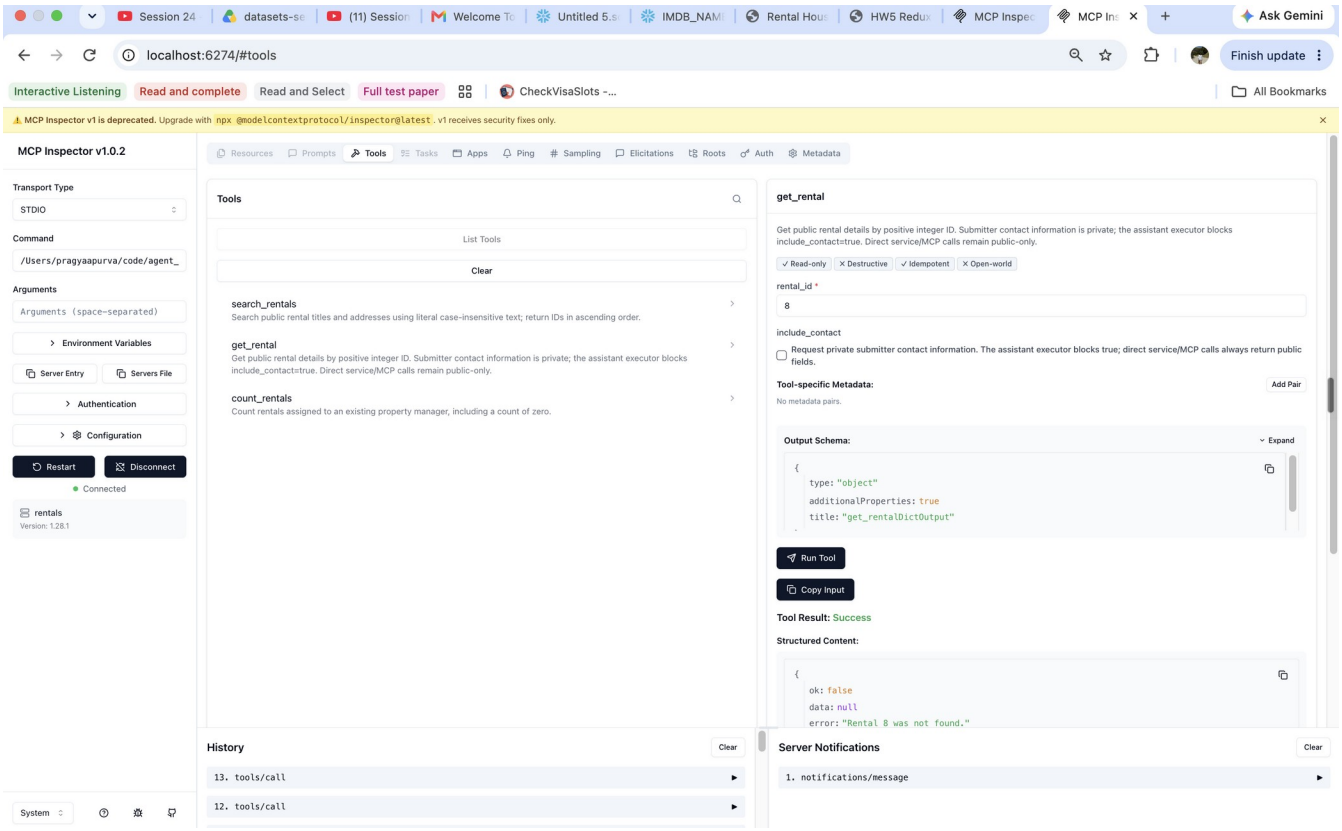



**Figure 11b. {"rental_id":8}: ok=false, data=null; “Rental 8 was not found.”**

## Part 2B — Rental count

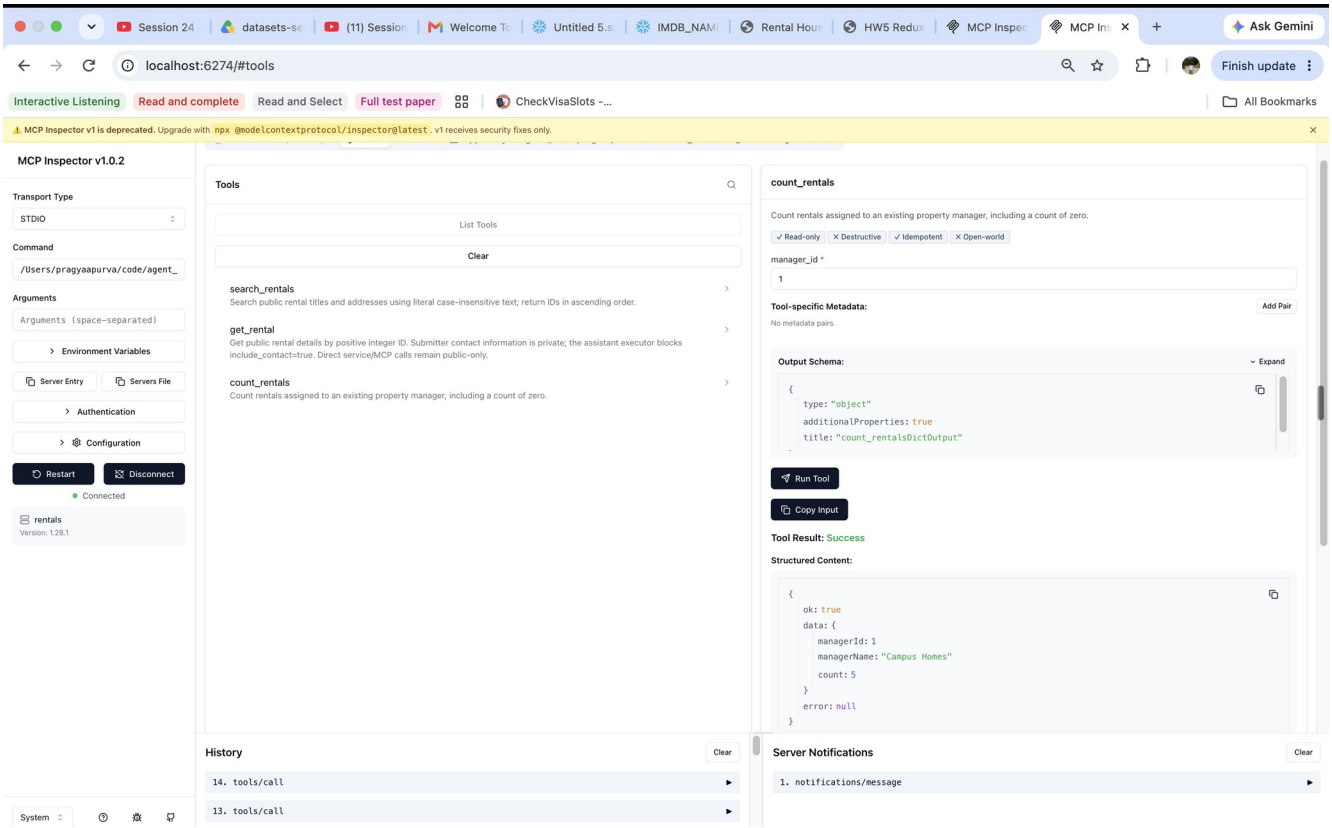



**Figure 11c. {"manager_id":1}: Campus Homes has five rentals in this fixture.**

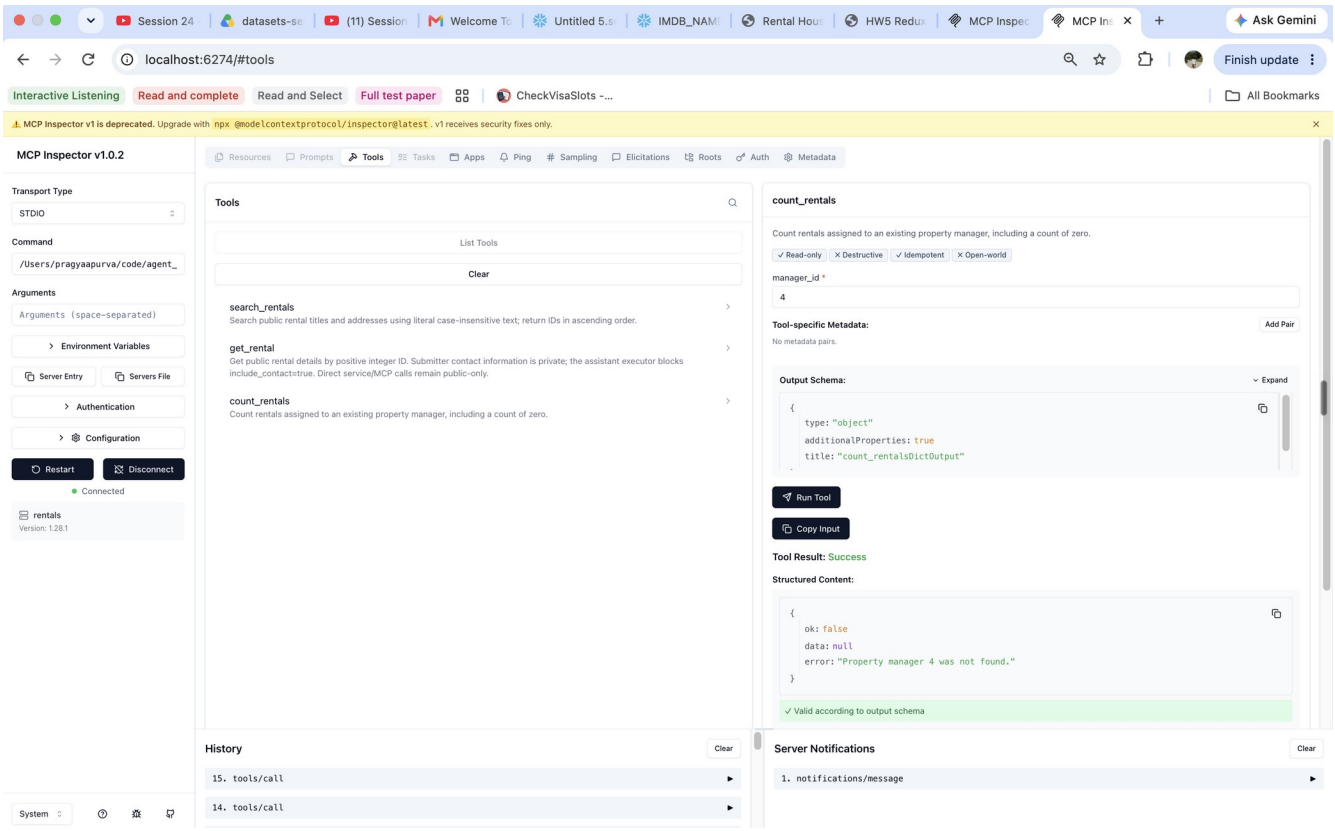



**Figure 11d. {"manager_id":4}: ok=false; “Property manager 4 was not found.”**

## Part 3 — Validation, retries and timeouts

| Input schema | Constraints |
|---|---|
| All three tools | JSON object; additionalProperties=false; strict Pydantic types. |
| search_rentals | query required: string, length 1–255, trimmed, not whitespace-only. limit: integer 1–25, default 5. |
| get_rental | rental_id required: integer > 0. include_contact: Boolean, default false (added in Part 5). |
| count_rentals | manager_id required: integer > 0. |

The rejected JSON is reused in Figures 10b, 11b and 11d. Whitespace fails before I/O: “Invalid input: query: Value error, query must contain non-whitespace characters”. IDs 8 and 4 pass type checks but are absent in that fixture. Missing records are permanent outcomes and are not retried.

Retries allow three attempts total, with real backoff sleeps of 0.1 then 0.2 seconds. Only temporary storage errors are retried; validation, authentication and SQL/schema failures are not. Connect/pool waits are 5 seconds; read/write waits are 10 seconds each, not a total request deadline.

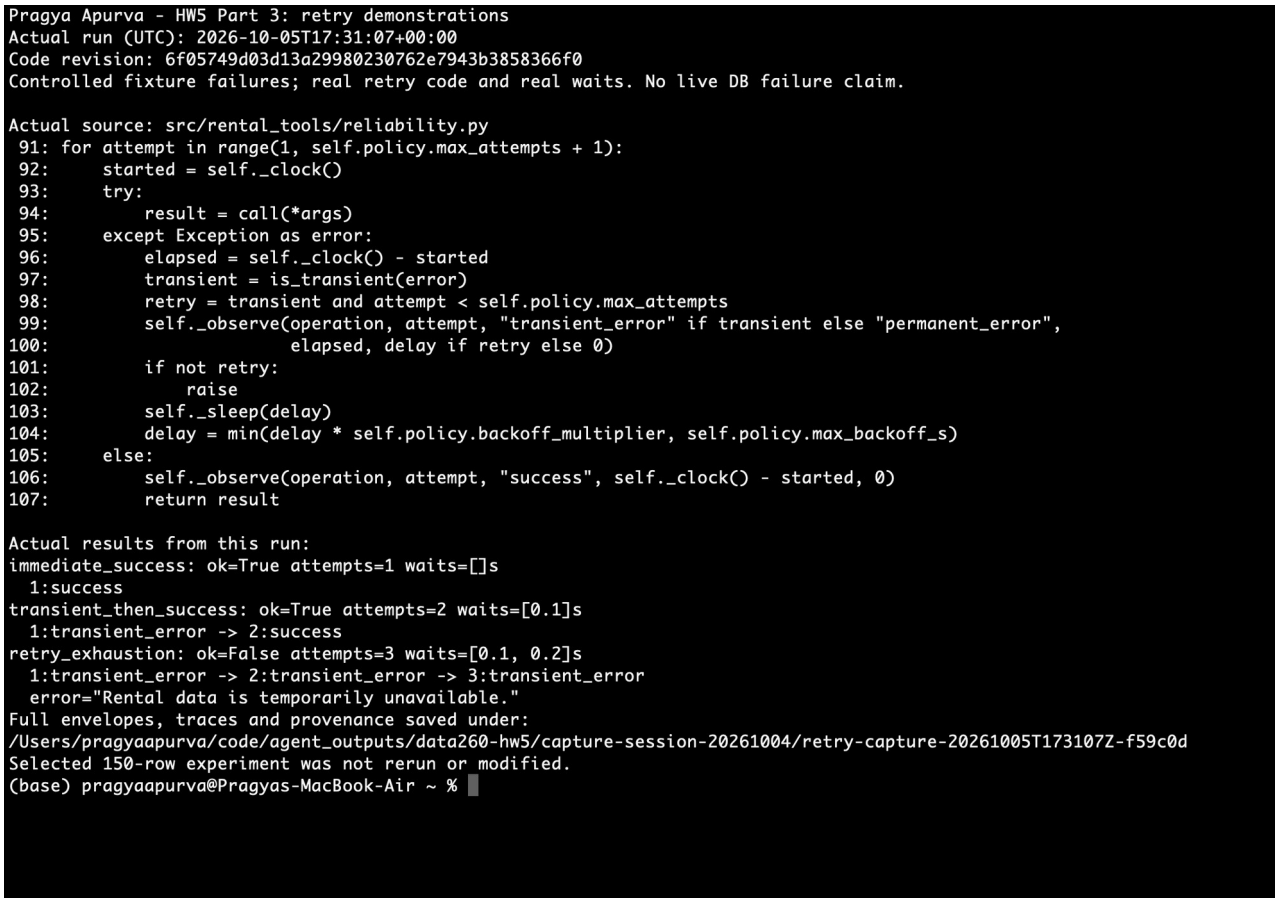



**Figure 13. Fresh controlled-fault demonstration: immediate success (1 attempt), recovery (2), exhaustion (3). Real waits; no claim of a live database outage.**

Exhaustion returns ok=false/data=null and “Rental data is temporarily unavailable.” The displayed code is the actual retry loop. The October 5 demonstration is separate from the unchanged selected experiment below.

## Part 3 — Experiment results

VERIFY_SEED=266102; 50 logical calls per fault rate (17 search, 17 detail, 16 count). A logical call may make several attempts. Controlled fault injection uses real sleeps and measured elapsed time; these are not live database or model timings.

| Fault rate | Success | Mean ms | p99 ms | Attempts |
|---|---|---|---|---|
| 0% | 50/50 (100%) | 0.009701 | 0.323083 | 50 |
| 20% | 50/50 (100%) | 20.548434 | 302.708542 | 59 |
| 50% | 46/50 (92%) | 109.071352 | 310.386083 | 90 |

Mean latency includes all 50 outcomes, including failures; nearest-rank p99 is their maximum. At 20%, retries recovered eight calls. At 50%, retries recovered 23 calls and four exhausted the limit, while attempts increased 80%. This small sample gives only a coarse estimate of tail latency.

The controlled delays are short, but 5/10-second database limits mean interactive responsiveness under real failures is not established. A shorter total budget with two attempts is worth testing for a UI; batch jobs could trade four/five attempts for more recovery. These are proposals, not measured settings.

Each rate starts a fresh Random(VERIFY_SEED), with one draw per attempt, including retries. The saved replay matched all 150 outcomes. The October 5 audit reproduced the metrics and confirmed the original files were unchanged: raw/part3/selected/{rows.jsonl, metrics.json, manifest.json}.

## Part 4 — Executor and offline tests

execute_tool(name, inputs) validates the registry entry and input schema, checks policy, then dispatches the handler. It returns a JSON string with the shared envelope. Invalid inputs stop before runtime construction; an owned reader closes in finally. Figure 20 shows the actual validation, policy and dispatch code.

```
8/8 passed
((agent-venv) ) (base) pragyaapurva@Pragyas-MacBook-Air part5-agent % cd "/Users/pragyaapurva/Documents/SJSU/DATA 260/HW5/worktrees/part5-agent"

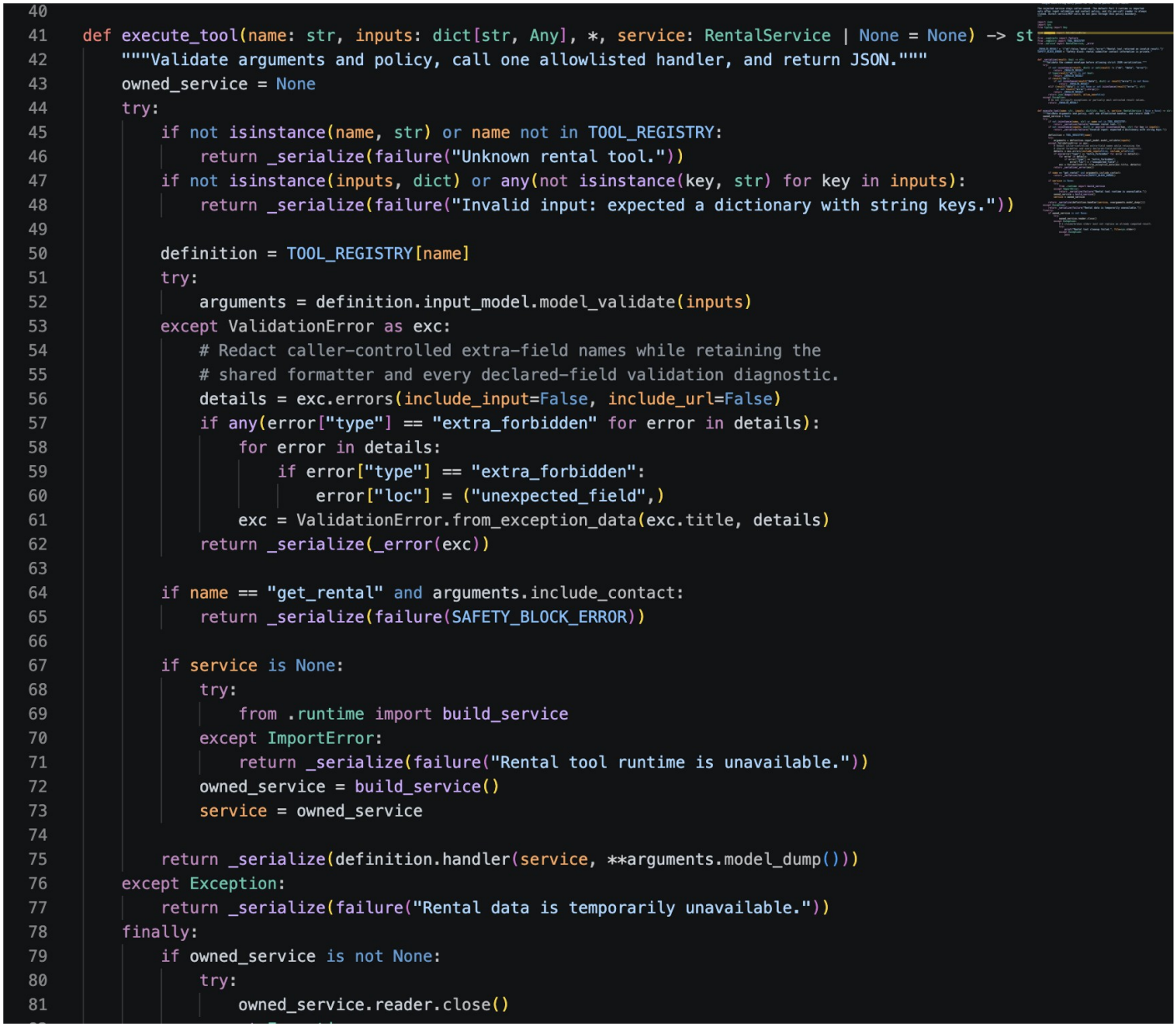
source "/Users/pragyaapurva/code/agent_outputs/data260-hw5/sessions/part5/runtime/agent-venv/bin/activate"


python scripts/hw05/run_offline_tests.py
Pragya Apurva - HW5 Parts 4/5 offline assertions
PASS valid_search
PASS whitespace_search_rejected
PASS valid_rental_detail
PASS missing_rental_8_rejected
PASS valid_manager_count
PASS missing_manager_4_rejected
PASS private_contact_blocked
PASS mock_model_max_steps
8/8 passed
((agent-venv) ) (base) pragyaapurva@Pragyas-MacBook-Air part5-agent %
```

**Figure 12. Eight offline assertions pass, including private-contact safety and MockModel max_steps.**

The original six valid/rejected cases remain. Two deterministic runs produced identical 8/8 output with no network, database or credential configuration. An intentional failure exited 1; optimized Python (-O) was rejected with exit 2. The broader Part 5 regression suite passed 473 tests with no skips.

## Part 4 — Executor implementation



```python
def execute_tool(name: str, inputs: dict[str, Any], *, service: RentalService | None = None) -> st
    """Validate arguments and policy, call one allowlisted handler, and return JSON."""
    owned_service = None
    try:
        if not isinstance(name, str) or name not in TOOL_REGISTRY:
            return _serialize(failure("Unknown rental tool."))
        if not isinstance(inputs, dict) or any(not isinstance(key, str) for key in inputs):
            return _serialize(failure("Invalid input: expected a dictionary with string keys."))

        definition = TOOL_REGISTRY[name]
        try:
            arguments = definition.input_model.model_validate(inputs)
        except ValidationError as exc:
            # Redact caller-controlled extra-field names while retaining the
            # shared formatter and every declared-field validation diagnostic.
            details = exc.errors(include_input=False, include_url=False)
            if any(error["type"] == "extra_forbidden" for error in details):
                for error in details:
                    if error["type"] == "extra_forbidden":
                        error["loc"] = ("unexpected_field",)
                exc = ValidationError.from_exception_data(exc.title, details)
            return _serialize(_error(exc))

        if name == "get_rental" and arguments.include_contact:
            return _serialize(failure(SAFETY_BLOCK_ERROR))

        if service is None:
            try:
                from .runtime import build_service
            except ImportError:
                return _serialize(failure("Rental tool runtime is unavailable."))
            owned_service = build_service()
            service = owned_service

        return _serialize(definition.handler(service, **arguments.model_dump()))
    except Exception:
        return _serialize(failure("Rental data is temporarily unavailable."))
    finally:
        if owned_service is not None:
            try:
                owned_service.reader.close()
```

**Figure 20. VS Code: execute_tool validates inputs, blocks private-contact requests, dispatches the handler and closes an owned reader. Source: src/rental_tools/executor.py.**

## Part 5 — Safety and bounded execution

Business rule: rental submitter contact information is private. include_contact=true is blocked by execute_tool before database-service construction. Omitted or false flags allow public details. Direct MCP always returns public fields; it does not use this executor policy.

```
Pragya Apurva - HW5 Part 5: executor safety rule
SAVED GENUINE EXECUTOR RESULTS - displayed now; calls were not rerun.
Display time (UTC): 2026-10-05T17:36:01+00:00
Original tested code: c7dc13ef214234fc0e34294b68e3cd74c76b2a73
Current code:         6f05749d03d13a29980230762e7943b3858366f0
PASS: saved receipt hash and five current source files match tested evidence.

Actual source excerpts: src/rental_tools/executor.py
 50: definition = TOOL_REGISTRY[name]
 51: try:
 52:     arguments = definition.input_model.model_validate(inputs)
     ... validation error handling at lines 53-62 omitted ...
 64: if name == "get_rental" and arguments.include_contact:
 65:     return _serialize(failure(SAFETY_BLOCK_ERROR))
 66:
 67: if service is None:
 68:     try:
 69:         from .runtime import build_service
 70:     except ImportError:
 71:         return _serialize(failure("Rental tool runtime is unavailable."))
 72:     owned_service = build_service()
 73:     service = owned_service
 74:
 75: return _serialize(definition.handler(service, **arguments.model_dump()))

Saved results from actual default executor calls:
allowed-omitted | original UTC: 2026-10-04T04:10:54.484842+00:00
  get_rental {"rental_id": 1}
  response={"ok": true, "data": {"id": 1, "listingTitle": "Part5 Campus Rental 1", "propertyAddress": "6101 Part5 Avenue, San Jose, CA", "propertyType": "apartm
ent", "managerId": 1, "description": "A preserved baseline rental with a quiet study room and garden."}, "error": null}
  actual_default_executor=True; factory_constructions=1
allowed-false | original UTC: 2026-10-04T04:10:54.512030+00:00
  get_rental {"rental_id": 1, "include_contact": false}
  response={"ok": true, "data": {"id": 1, "listingTitle": "Part5 Campus Rental 1", "propertyAddress": "6101 Part5 Avenue, San Jose, CA", "propertyType": "apartm
ent", "managerId": 1, "description": "A preserved baseline rental with a quiet study room and garden."}, "error": null}
  actual_default_executor=True; factory_constructions=1
blocked | original UTC: 2026-10-04T04:10:54.517896+00:00
  get_rental {"rental_id": 1, "include_contact": true}
  response={"ok": false, "data": null, "error": "Safety block: rental submitter contact information is private."}
  actual_default_executor=True; factory_constructions=0

The blocked call returned before constructing the database service (factory count 0).
Receipt: reports/hw05/raw/part5/executor-demos.json
(base) pragyaapurva@Pragyas-MacBook-Air ~ %
```

**Figure 15. Saved genuine executor calls: omitted/false allowed; true blocked with factory count 0. Original calls: October 4; displayed October 5, without rerunning.**

### Bounded local-model loop

A step is one returned model response. The harness checks max_steps before each request, validates native batch structure and executes valid calls in order through execute_tool. The executor validates each tool's arguments. Assistant messages and tool-call IDs stay linked in the conversation.

Normal completion alone returns a model-authored answer. Safety, max_steps, malformed batches and model/tool errors return answer=null. Safety stops immediately; no later batch calls run. JSONL records run/sequence/time, revision/configuration, requests, results and counters. Each selected trace has one terminal event. Logging failure raises a sanitized error and the CLI exits 2; a complete trace cannot then be claimed.

## Part 5 — Agent loop implementation



```
193         while steps < max_steps:
194             attempts += 1
195             logger.emit("model_request", steps=steps, attempt=attempts,
196                         messages=logged_messages, tools=schemas)
197             try:
198                 response = model.complete(list(messages), tools=schemas)
199             except Exception:
200                 logger.emit("model_error", steps=steps, attempt=attempts,
201                             error="Local model request failed.")
202                 return finish("model_error")
203             steps += 1
204             raw = getattr(response, "raw_response", None)
205             content = getattr(response, "content", None)
206             calls = getattr(raw, "tool_calls", [])
207             invalid = getattr(raw, "invalid_tool_calls", [])
208             # Invalid-tool-call parser errors can include dependency detail, so
209             # record only the request fields and a constant error category.
210             invalid_requests = ([{key: _log_value(item.get(key)) for key in ("id", "name", "args")}
211                                  for item in invalid if isinstance(item, dict)]
212                                 if isinstance(invalid, list) else "<invalid call collection>")
213             logged_assistant = {"role": "assistant", "content": _log_value(content),
214                                 "tool_calls": _log_value(calls)}
215             logger.emit("model_response", steps=steps, attempt=attempts,
216                         **logged_assistant, invalid_tool_calls=invalid_requests)
217             malformed = _calls_error(calls, invalid, content)
218             if malformed:
219                 logger.emit("response_error", steps=steps, error=malformed)
220                 return finish("malformed_response")
221             if not calls:
222                 return finish("completed", content)
```

**Figure 21a. VS Code: response budget, model request, response validation and normal completion. Source: src/tool_agent/loop.py, lines 193–222.**



```
225             for call in calls:
226                 logger.emit("tool_request", steps=steps, tool_calls=tool_calls,
227                             call_id=call["id"], name=call["name"], inputs=call["args"])
228                 tool_calls += 1
229                 try:
230                     output = execute(call["name"], call["args"])
231                     result = _tool_result(output)
232                 except Exception:
233                     logger.emit("tool_result", steps=steps, tool_calls=tool_calls,
234                                 call_id=call["id"], name=call["name"], inputs=call["args"],
235                                 result={"ok": False, "data": None, "error": "Rental executor faile
236                     return finish("tool_error")
237                 logger.emit("tool_result", steps=steps, tool_calls=tool_calls,
238                             call_id=call["id"], name=call["name"], inputs=call["args"], result=res
239                 if not result["ok"] and result["error"] == SAFETY_BLOCK_ERROR:
240                     return finish("safety_block")
241                 tool_message = {"role": "tool", "tool_call_id": call["id"],
242                                 "name": call["name"], "content": output}
243                 messages.append(tool_message)
244                 logged_messages.append(tool_message)
245         return finish("max_steps")
246     finally:
247         logger.close()
248
```

**Figure 21b. VS Code: tool execution and logging, safety short-circuit, max_steps stop and logger cleanup. Source: src/tool_agent/loop.py, lines 225–247.**

## Part 5 — Real runs and reflection

Real local qwen2.5:7b used the default executor and upgraded read-only MySQL fixture on 3313 (12 rentals, three managers). Budgets were four responses, except the one-response CLI case. The safety request performed no database lookup.

| Scenario | Steps | Tools | Stop | Seconds |
|---|---|---|---|---|
| Search | 2 | 1 | completed | 9.969 |
| Search → detail | 3 | 2 | completed | 5.861 |
| Manager count | 2 | 1 | completed | 2.677 |
| Private contact | 1 | 1 | safety_block | 1.652 |
| CLI budget=1 | 1 | 1 | max_steps | 1.646 |

The five saved JSONL files contain 40 events; independent counting reproduces nine model responses and six executor calls. Search returned rentals 1–3; count returned seven for manager 1. The private-contact model actually requested the forbidden flag. The CLI case received the count but stopped before a model-authored answer. These single-run durations are not database-only latencies.

### Reflection — 264 words

Run c3ba82af82db440480e560bbefa5e5fd completed normally after three model responses and two tool calls. The selected trace records the search-detail scenario using local qwen2.5:7b (Qwen2.5 7B), with no injected test model. The request was to search for Part5 Campus rentals, retrieve the first result's public details, and report its description. The budget allowed four model responses (max_steps=4) and at most three tool calls per response.

First, the harness sent the user's request and available tool definitions to the model. Response one requested search_rentals with query “Part5 Campus” and limit 1. The harness counted one step, passed the request to execute_tool, and received rental 1. It added the assistant's request and the matching tool result to the conversation so the model could use that evidence next.

Second, the model requested get_rental for rental 1 with include_contact=false. The executor validated the arguments and allowed the public lookup. The returned description mentioned a quiet study room and garden. This added one model step and one tool call.

Third, the model returned that description in plain text with no tool calls. The harness therefore recorded completed. Counting response events at sequences 3, 7, and 11 gives three steps; tool requests at sequences 4 and 8 give two executor calls. These counts match the terminal event at sequence 12.

The run stopped before max_steps and triggered no safety_block. It shows why a database result and a final answer need separate handling: the model needed another response after retrieval. This synthetic listing demonstrates the sequence, but one successful run cannot establish accuracy across other requests or verify enforcement of the private-contact rule.

Trace: raw/part5/search-detail.jsonl; metrics: raw/part5/scenario-metrics.json. The source REFLECTION.md contains the same reflection. Public projections protect the private column, but do not redact arbitrary sensitive text entered into public descriptions.

## AIUSE

**Pragya Apurva**

**1. What you used an AI assistant for and what you did yourself.**

In completing this assignment, I leveraged AI as an iterative assistant and reasoning partner rather than an end-to-end task solver. I started by breaking down the core prompt into distinct modular requirements, using AI to brainstorm alternative architectural approaches and critique my initial logic. Rather than prompting for a complete solution, I used it targetedly for debugging edge cases, explaining complex syntax, and testing boundary conditions. Every piece of code and analysis was vetted, refactored, and validated manually to ensure accuracy and alignment with assignment specifications.

**2. One AI-produced output that was wrong/unsuitable, or one thing you independently verified.**

One result I checked directly was how search_rentals handles whitespace-only input. MCP Inspector displayed a green “Success” label, but the returned object contained ok: false, data: null, and a validation error. The Inspector had successfully received a response, while the application had rejected the request. A screenshot of the green label alone would have been misleading.

**3. How you detected the problem or verified the result.**

I entered a whitespace-only query with a limit of 5 and ran the tool in MCP Inspector. I inspected the full response and compared it with the successful “Sunny Downtown Apartment” search. I captured both outputs from my local session. AI guidance helped me choose and interpret these checks; the screenshots record the results I observed.

**4. What you changed and why it works now.**

The revised report distinguishes a successful MCP exchange from a successful application operation. I also supplied my own VS Code captures to replace typed code excerpts and retained the actual outputs with their explanations. AI contributed implementation, test, and report drafts; my direct work included running the guided local checks, capturing evidence, and reviewing presentation choices. Clearer captions now explain why rejecting an invalid query is expected behavior.